基于一维投影的简单，可拓展且有效的聚类方法

摘要

聚类是无监督机器学习中的一个基本问题，在数据分析领域有着广泛的应用。流行的聚类算法，如 Lloyd 算法和 k-means++ 算法，在将 d 维空间中的 n 个点（由 $n \times d$ 矩阵 X 表示）聚类为 k 个簇时，往往需要 $\Omega(ndk)$ 的时间。在 k 值中等或较大的应用场景中，这个乘法因子 k 会导致巨大的计算开销。我们介绍了一种简单的随机聚类算法，并证明了对于任意的 k，该算法的期望运行时间为 $O(nnz(X) + nlogn)$。其中，$nnz(X)$ 表示输入数据集 X 中非零元素的总数，其上界为 nd，但在稀疏数据集中该数值可能显著更小。我们证明了该算法在任意输入数据集上，针对 k-means 目标函数均能实现 $\tilde{O}(k^4)$ 的近似比。我们还认为本文的理论分析具有独立的研究价值，因为我们展示了 k-means 算法的近似比在一类投影变换下能够大致保持不变，并且证明了 k-means++ 的初始化（seeding）过程可以在一维空间中 $O(nlogn)$ 的期望时间实现。最后，实验结果表明，与之前针对此类任务的最先进（state-of-the-art）方法相比，我们的聚类算法在运行时间和聚类质量之间提供了新的权衡（tradeoff）。

1. 引言

聚类是数据分析中一种重要且强大的工具，在计算机视觉和计算生物学中有着广泛的应用，同时也是无监督机器学习中的基本问题之一。在大规模应用中，数据集通常包含数十亿个高维数据点。将相似的数据点归组为簇（Clusters）对于理解和组织数据集至关重要。鉴于其实际重要性，设计高效且有效的聚类算法几十年来一直吸引着众多研究者的关注。

针对 k-means 聚类问题，最流行[52]的算法之一是 Lloyd 算法。该算法试图在空间中定位 k 个中心，以最小化数据集中各点到其最近中心的距离平方和（我们称之为“k-means 代价”）。虽然寻找最小化该目标函数的中心点集是 NP-难（NP-hard）问题[3]，但在实践中，我们可以使用 Lloyd 迭代算法找到高质量的中心点集。Lloyd 算法维护一组 k 个中心，通过将点分配给 k 个簇之一（根据最近的中心），然后将中心重新定义为簇内点的质心，以此迭代地更新它们。该算法需要一个良好的初始中心集以获得高质量的聚类结果并实现快速收敛。在实践中，通常使用 k-means++ 算法[6]（一种随机初始化过程）来选择初始的 k 个中心。K-means++ 在期望上能达到 $O(logk)$ 的近似比，Lloyd 算法的每次迭代都在此基础上进一步优化。除了有效性之外，这些算法描述简单且易于实现，这也是它们广受欢迎的原因。

然而，这些算法的缺点在于无法很好地扩展到海量数据集。Lloyd 算法一次迭代的标准实现需要计算数据集中每个点到每个中心的距离，导致 $\Theta(ndk)$ 的运行时间。同样，k-means++ 初始化过程的标准实现需要从所谓的 D^2分布（详见附录 A）中产生 k 个样本。维护该分布需要在选择每个样本后遍历整个数据集。生成 k 个中心会导致 $\Theta(ndk)$ 的运行时间。即使对于中等大小的 k 值，对整个数据集进行 k 次遍历也是极其昂贵的。

大规模 k-means 聚类的一个特别相关的应用是近似最近邻搜索[66]（Approximate Nearest Neighbor Search）（例如，在乘积量化[44] 和构建倒排文件索引[16] 中）。在这些应用中，k-means 聚类用于通过将向量映射到其最近的中心来压缩整个数据集，这导致了具有大 k 值（数百或数千量级）的十亿级规模聚类问题。其他需要大量中心的大规模数据集应用可能包括垃圾邮件过滤、近似重复检测以及压缩或协调任务。这些大规模场景需要新的算法思路，从而引发了以下挑战：

$$\text{我们能否设计一种简单、实用的 k-means 算法，其运行时间大致为}O(nd)\text{（与 }k\text{ 无关），并能产生高质量的簇？}$$

鉴于其在理论和实践中的重要性，大量工作致力于研究快速 k-means 聚类算法。我们在下文中总结了几种方法及其优缺点，以突显本文工作在文献中的定位：

- A. **标准 k-means++**：这是我们的标准基准。**优点**：保证 $O(logk)$ 的近似比；输出中心点以及数据点到中心的分配。**缺点**：运行时间为$O(ndk)$，在大规模应用中过于昂贵。
- B. **使用近似最近邻搜索**：可以使用近似最近邻搜索技术（而非每次迭代的暴力搜索）来加速 k-means++。**优点**：具有可证明保证的算法，可获得$O\varepsilon(logk)$的近似比。**缺点**：运行时间为 $\tilde{O}(nd + (nlog(\Delta))^{1+\varepsilon})$，依赖于数据集相关参数 Δ（输入点之间最大距离与最小距离的比率）。这些技术在算法上很复杂，并引入了额外的多对数因子（隐藏在 $\tilde{O}(\cdot)$中），使得实现显著复杂化。
- C. **近似 D^2分布**：文献提出了加速 Lloyd 算法初始化过程或生成快速核心集（Coresets，下文详述）的算法。**优点**：这些算法很快，只需以 $O(nd)$ 时间遍历一次数据集。（对于 ，运行时间中有额外的加性 项）。**缺点**：其近似保证在性质上弱于 k-means 聚类的近似。它们引入了额外的加性近似误差（additive approximation error），该误差随整个数据集的方差增长（这可能导致任意大的误差；见第 3 节）。这些算法输出一组 k 个中心，但不输出簇的分配。简单地生成分配结果仍需$O(ndk)$ 时间。

核心集（Coresets）： 从高层次来看，核心集是一种数据集缩减机制。将 $\mathbb{R}^d$ 空间中包含 n 个点的大型数据集 X 提炼为一个显著更小的、包含 m 个点的（加权）数据集 Y，称为“核心集”，作为 X 的良好代理。即，任何 k 个中心在 Y 上的聚类代价近似等于这些中心在 X 上的代价。我们推荐读者参考[9,35]以获取关于核心集的最新综述。重要的是，核心集的构建（具有可证明的乘性近似保证）需要对原始数据集 X 进行初始的近似聚类。因此，任何快速的 k-means 聚类算法都会自动加速任何使用核心集进行聚类的算法流程——展望未来，我们将展示我们的算法如何在不牺牲近似质量的情况下显著加速核心集的构建。

理论结果： 我们提出了一种简单的随机聚类算法，在不对数据做任何假设的情况下，对其运行时间和近似比给出了可证明的保证。它的优点在于速度快（类似上述类别 C 的算法），同时实现了乘性误差保证而没有额外的加性误差（类似上述类别 B 的算法）。

- 无论 k 大小如何，算法运行时间均为 $O(nd + n\log n)$。它遍历数据集一次以执行数据缩减（贡献了 nd 因子），加上额外的 $O(n\log n)$ 项用于在缩减后的数据上求解 k-means，从而产生 k 个中心和簇分配。在稀疏输入数据集上，nd 项变为 $\mathrm{nnz}(X)$，即数据集中非零元素的数量。因此，我们的算法在稀疏矩阵上的运行时间为 $O(\mathrm{nnz}(X) + n\log n)$。
- 该算法与 k-means++ 算法一样简单，但效率显著更高。我们证明的近似比为 $\mathrm{poly}(k)$（k 的多项式级），虽然比 k-means++ 实现的 $O(\log k)$ 差，但它是纯乘性的（参见下文关于将其改进为 $O(\log k)$ 的备注）。它不会产生 [8, 7, 10] 中快速算法所带来的额外加性误差。
  我们的算法将输入点投影到一个随机的一维空间，并在投影后运行高效的 k-means++ 初始化。为了获得近似保证，我们分析了投影后获得的近似比如何传递回原始点（引理 2.5）。我们通过在一维中高效实现 k-means++ 初始化并通过势能函数参数分析运行时间（引理 2.4）来界定算法的运行时间。我们的算法不仅适用于 k-means，还适用于其他将距离的 z 次幂求和的聚类目标（对于一般的 $z \geq 1$），并且我们在运行时间和近似比上的保证可以平滑地扩展到这些设置中。

将近似比从 $\mathrm{poly}(k)$ 改进为 $O(\log k)$： $\mathrm{poly}(k)$ 的近似比看起来可能比 k-means++ 可达到的 $O(\log k)$ 近似差得多。然而，我们可以通过在运行时间中增加一个加性的 $O(\mathrm{poly}(kd)\cdot\log n)$ 项将其改进为 $O(\log k)$。利用附录 A.2 中讨论的先前结果（特别是定理 A.2），一个乘性的 $\mathrm{poly}(k)$ 近似足以构建大小为 $\mathrm{poly}(kd)$ 的核心集，并在该核心集上运行 k-means++。构建核心集很简单，耗时 $\mathrm{poly}(kd)\cdot\log n$（通过从适当分布中采样）；在核心集上运行 k-means++ 耗时 $\mathrm{poly}(kd)$（不依赖于 n）。将我们的算法与核心集结合，我们在 $O(\mathrm{nnz}(X)) + O(n\log n) + \mathrm{poly}(kd)\cdot\log n$ 的时间内获得了 $O(\log k)$ 的近似。值得注意的是，[8, 7, 10] 的加性近似方法无法实现这些保证。

2. 算法概述与证明技术

我们的算法称为 **PRONE** (PRojected ONE-dimensional clustering，投影一维聚类)，其通过将数据随机投影到一维空间，对投影后的（标量）数值进行排序，并在这些数值上运行 k-means++ 初始化策略。凭借其简洁性，该算法具有良好的可扩展性，且在海量数据集聚类上非常有效。更形式化地讲，PRONE 接收一个包含 n 个点的 $\mathbb{R}^d$ 空间数据集作为输入，以及一个参数 $k \in \mathbb{N}$（期望的簇数量），并按如下步骤执行：

1. 从标准高斯分布中采样一个随机向量 $v \in \mathbb{R}^d$，并将数据点沿 v 的方向投影到一维。即，我们通过遍历一次数据计算 $x'_i = \langle x_i, v\rangle \in \mathbb{R}$，耗时 $O(\mathrm{nnz}(X))$，有效地将数据集缩减为一维点集 $x'_1, \ldots, x'_n \in \mathbb{R}$。
2. 在 $x'_1, \ldots, x'_n$ 上运行 k-means++ 初始化，获得 k 个索引 $j_1, \ldots, j_k \in [n]$（指示选定的中心 $x'_{j_1}, \ldots, x'_{j_k}$）以及一个分配 $\sigma : [n] \to [k]$，将点 x'_i 分配给中心 $x'_{j_{\sigma(i)}}$。尽管 k-means++ 初始化在一维中通常需要 $O(nk)$ 时间，但利用点是一维的事实，我们给出了一种高效实现，其期望运行时间为 $O(n\log n)$，与 k 无关。详细的算法描述见附录。
3. 一维 k-means++ 算法产生一组 k 个中心 $x_{j_1}, \ldots, x_{j_k}$，以及将每个点 x_i 映射到中心 $x_{j_{\sigma(i)}}$ 的分配 σ。对于每个 $\ell \in [k]$，我们将簇 ℓ 的聚类中心更新为所有分配给 x_{j_ℓ} 的点的质心。

虽然算法直观明了，但主要的技术难点在于分析。特别是，我们的分析 (1) 界定了步骤 1 中一维投影产生的近似损失，以及 (2) 证明了我们可以以 $O(n\log n)$ 的期望时间实现步骤 2，而不是 $O(nk)$ 时间。我们在以下定理中总结了理论贡献。

定理 2.1： 算法 PRONE 在任意数据集 $X = \{x_1, \ldots, x_n\} \subset \mathbb{R}^d$ 上的期望运行时间为 $O(\mathrm{nnz}(X) + n\log n)$。此外，对于任意 $\delta \in (0, 1/2)$ 和任意数据集 X，算法以至少 $1-\delta$ 的概率在 $O(\mathrm{nnz}(X) + n\log(n/\delta))$ 时间内运行。

定理 2.2： 算法 PRONE 针对 k-means 目标函数以至少 0.9 的概率实现 $\tilde{O}(k^4)$ 的近似比。

据我们所知，PRONE 是第一个对于任意 k 都能在 $O(nd + n\log n)$ 时间内运行的 k-means 算法。正如在关于改进竞争比（competitive ratio）的段落中所述，利用核心集（coreset）的两阶段方法，我们可以从前两个定理中得出以下推论：

推论 2.3： 通过将 PRONE 作为定理 A.2 中的 α-近似算法使用，并在生成的核心集上运行 k-means++，我们获得了一个近似比为 $O(\log k)$ 的算法，其运行时间为 $O(\mathrm{nnz}(X) + n\log n + \mathrm{poly}(kd)\log n)$，且具有常数成功概率。

由于篇幅限制，我们理论结果的所有证明均推迟至附录中。在附录中，我们还将这些结果从 k-means 推广到了对一般 $z \geq 1$ 的欧几里得距离 z 次幂求和的聚类目标。以下小节对我们为证明上述主要定理而开发的主要技术进行了高层次的概述。

2.1 一维中的高效初始化 (Efficient Seeding in One Dimension)

k-means++ 初始化过程包含 k 次迭代，每次迭代采样一个新的中心。由于新中心可能需要更新 $\Omega(n)$ 个距离以维护 D^2 分布（该分布以与最近中心的距离成正比的概率采样每个点），朴素分析会导致 $O(nk)$ 的运行时间。证明定理 2.1 的一个关键要素是表明，对于一维数据集，k-means++ 仅需进行 $O(n\log n)$ 次更新，与 k 无关。

引理 2.4： k-means++ 初始化过程在一维中可以以 $O(n\log n)$ 的期望时间实现。此外，对于任意 $\delta \in (0, 1/2)$，以至少 $1-\delta$ 的概率，该实现运行时间为 $O(n\log(n/\delta))$。

证明的直觉如下：由于点是一维的，我们始终保持它们的排序状态。此外，每个数据点 x_i 将维护其中心分配以及到最近中心的距离 p_i。通过在排序的点上构建二叉树（内部节点维护 p_i^2 的和），可以很容易地在 $O(\log n)$ 时间内从 D^2 分布中采样一个新中心。困难在于，添加一个新中心可能会导致多个点 x_i 的 p_i 值发生变化，因此挑战在于界定这些值被更新的次数（见下图 1）。

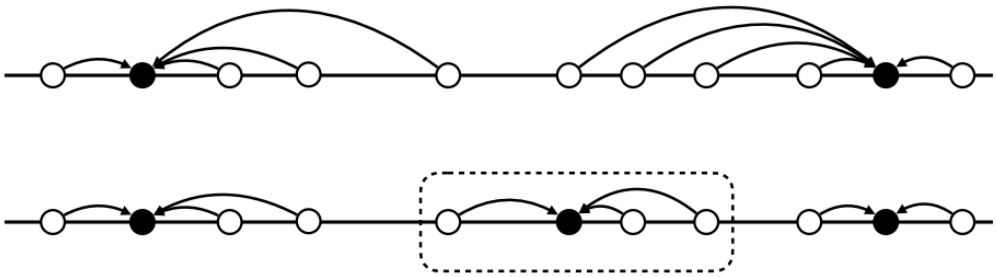


图 1： 从上到下，选择了一个新中心（黑色圆圈）。每个点都有一个箭头指向其最近的中心。虚线框中的点是需要更新的点。

为了界定总运行时间，我们利用了一维结构。观察到，对于一个新中心，被更新的点位于新选中心周围的一个连续区间内。一旦选中一个中心，算法向左和向右扫描点，直到遇到一个不需要更新的点。该点表明其另一侧的点也不需要更新，因此我们可以不必检查所有 n 个点（见图 1）。有些出人意料的是，当从 D^2 分布中采样中心时，每个点被更新的期望次数仅为 $O(\log n)$，这意味着总更新次数的期望上界为 $O(n \log n)$。关于每个点被更新 $O(\log n)$ 次这一事实的分析是非平凡的，并使用了一个精心设计的势能函数（引理 C.5）。

2.2 来自一维投影的近似保证

我们对定理 2.2 的证明建立在一系列研究聚类问题的随机降维工作基础之上 [14, 21, 11, 53]。先前的工作研究了用于精确 $(1 \pm \epsilon)$-近似的随机降维。我们的视角略有不同；我们限制自己仅使用**一维投影**，并给出了失真（distortion）的上界。

对于任何数据集 $x_1, \ldots, x_n \in \mathbb{R}^d$，投影到一个随机的低维空间会以可预测的方式影响投影点之间的成对距离——Johnson-Lindenstrauss 引理就是一个典型的例子，它投影到 $O(\log n)$ 维。然而，当仅投影到**一维**时，成对距离会受到显著影响（最高可达 $\mathrm{poly}(n)$ 因子）。因此，朴素的分析只能给出 k-means 的 $\mathrm{poly}(n)$ 近似。为了将 c-近似改进为 $O(\log k)$ 近似，需要一个大小约为 $\mathrm{poly}(c/\log k)$ 的核心集。当 c 是 n 的多项式时，由于数据点最多只有 n 个，这个界限就变得毫无意义。

然而，尽管许多成对距离严重失真，我们证明了 k-means 的代价仅受到 $\mathrm{poly}(k)$ 因子的影响。从高层次来看，这是因为 k-means 代价优化的是成对距离的总和（根据选定的聚类）。由成对距离给出的单个加数可能会发生显著变化，但总和不会。我们的证明遵循 [21] 的方法，该方法表明（粗略地说）k 个最优中心的成对失真足以论证 k-means 的代价。当投影到一维时，k 个最优中心将产生最大的成对失真 $\mathrm{poly}(k)$（因为在 k 个中心之间只有 $O(k^2)$ 个成对距离）。这允许我们将投影后的 r-近似解提升为原始点的 $O(k^4 r)$-近似解。

引理 2.5 (非正式)： 对于 $\mathbb{R}^d$ 中的任意点集 X，在选择标准高斯向量 $v \in \mathbb{R}^d$ 时，以下情况发生的概率至少为 0.9。设 $X' \subset \mathbb{R}$ 是 X 在 v 上的一维投影，则 X' 的任何 r-近似 k-means 聚类都会给出 X 的一个 $O(k^4 r)$-近似聚类，且具有相同的聚类划分。

3. 实验结果 (Experimental Results)

在本节中，我们概述了算法的实验评估。实验以两种不同的方式评估算法。对于每种方式，我们测量运行时间和所得解的 k-means 代价（点到分配中心的距离平方和）。（1）首先，我们将算法作为包含核心集构建的流程的一部分进行评估——这是我们算法的预期用例。（2）其次，我们单独评估算法的近似 k-means 聚类能力，并将其与 k-means++ [6] 进行比较。根据定理 2.1 和 2.2，我们预计我们的算法速度会快得多，但输出的分配代价较高。我们的目标是凭经验量化这些差异。

所有实验均在 Linux 上运行，使用配备 3.9 GHz 第 12 代 Intel Core i7 六核处理器和 32 GiB RAM 的笔记本电脑。所有算法均使用 C++ 实现，除非下文另有说明，否则使用 `blaze` 库对数据集执行矩阵和向量运算。我们在本提交的补充材料中提供了代码。

数据集 (Datasets)： 在我们的实验中，我们使用以下四个数据集：
KDD [47]： 2004 KDD 蛋白质同源性挑战赛的训练数据。该数据集包含 145,751 个观测值，具有 77 个实值特征。
Song [12]： 515,345 首歌曲的音色信息，每首歌曲有 90 个特征，用于年份预测。
Census [32]： 1990 年美国人口普查数据，包含 2,458,285 个观测值，每个观测值有 68 个分类特征。
Gaussian： 一个包含 240,005 个点的 4 维合成数据集。这些点是通过在每个轴上距离原点较大的正距离处放置标准正态分布并采样 30,000 个点生成的。然后将这些点镜像，使质心保持在原点。最后，在原点放置 5 个点。这是轻量级核心集 (lightweight coresets) [10] 的对抗性示例，因为它们不太可能采样到接近数据集均值的点。

3.1 核心集构建比较 (Coreset Construction Comparison)

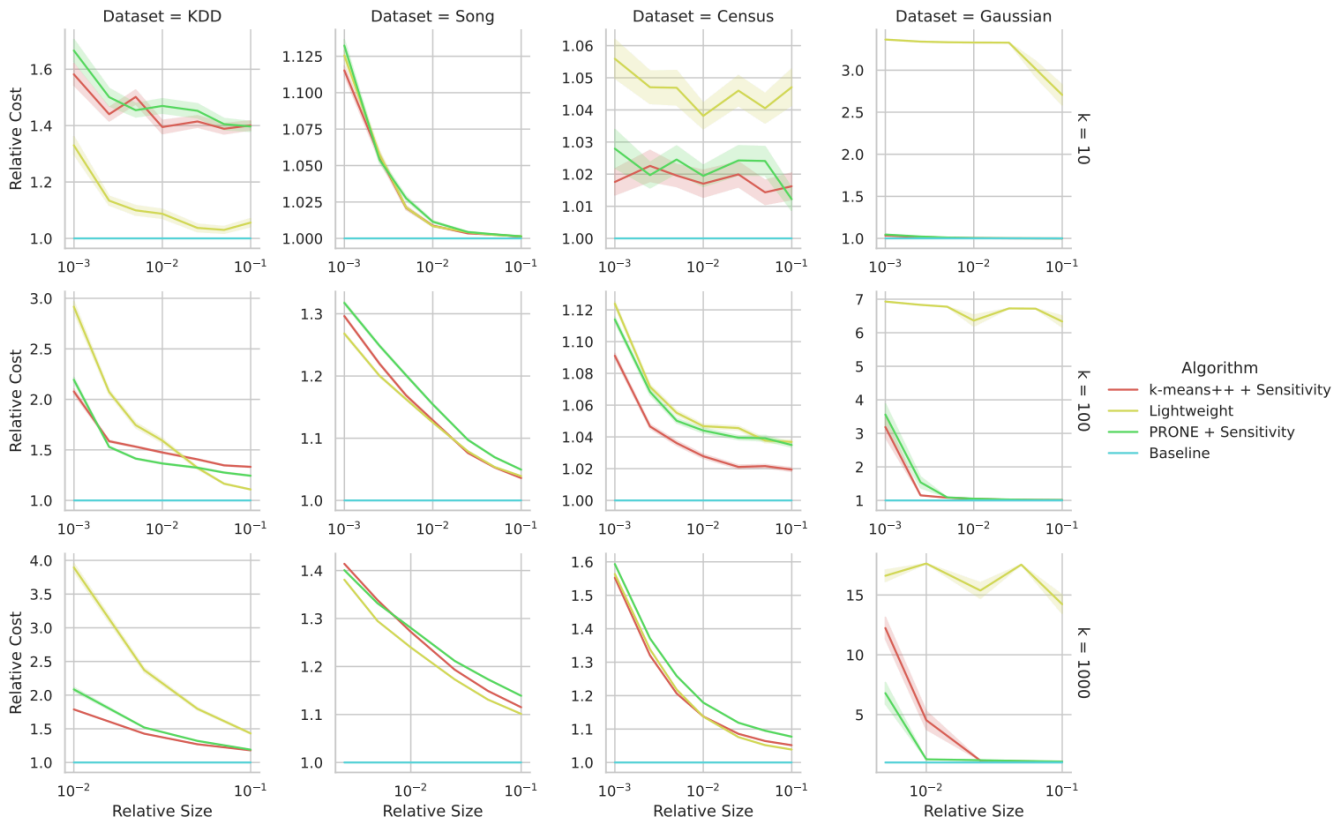
实验设置： 核心集构建（具有乘性近似保证）总是首先找到一个近似聚类，这构成了大部分工作。近似聚类定义了一个“敏感度采样分布”（我们在附录中对此进行了扩展，另请参见 [9]），通过从敏感度采样分布中重复采样来构建核心集。在我们的第一个实验中，我们评估了用于定义敏感度采样分布的初始近似算法的选择。我们比较了使用 k-means++ 和 PRONE。此外，我们还比较了 [10] 的轻量级核心集 (Lightweight)，它使用到质心的距离作为敏感度采样分布的近似值。在本节的其余部分，我们将使用 k-means++ 的敏感度采样称为 **Sensitivity**，将轻量级核心集称为 **Lightweight**。这三种算法都会生成一个核心集，实验将测量这三种算法的运行时间（表 1）和所得核心集的质量（图 2）。我们用 C++ 实现了所有算法。

一旦为每个算法构建了核心集，我们就通过计算聚类核心集时找到的中心的代价来评估核心集的质量（参见定义 A.1）。我们运行 `scikit-learn` 库 [57] 中 Lloyd 的 k-means 算法的最先进实现，使用默认配置（重复 15 次并报告平均代价以减少方差）。将所得核心集的质量与（计算昂贵的）基线进行比较，基线是在整个数据集上运行 `scikit-learn` 库中的 k-means++，然后运行默认配置的 Lloyd 算法（重复 5 次以减少方差）。

我们评估了 k 的各种选择 ($\{10, 100, 1000\}$) 以及各种相对大小的核心集，即数据集大小的 $\{0.001, 0.0025, 0.005, 0.01, 0.025, 0.05, 0.1\}$ 倍。我们使用的性能指标是：(1) 相对代价，即 Lloyd 算法在每个核心集上返回的 k-means 解的平均代价除以基线；(2) 核心集构建算法的运行时间。

核心集构建结果：相对代价 (Relative cost)： 图 2 显示了核心集大小（x 轴）与相对代价（y 轴）的关系。图 2 的每一“行”对应于 $k \in \{10, 100, 1000\}$ 的不同值，每一“列”对应于不同的数据集。回顾一下，前三个数据集（即前三列）是真实世界的数据集，第四列是合成的高斯数据集。我们注意到以下观察结果：

- 正如预期的那样，在所有真实世界数据集和所有 k 设置上，相对代价随着核心集大小的增加而降低。
- 在真实世界数据集中，每种核心集构建（Sensitivity, Lightweight 和我们的算法）的具体相对代价取决于数据集，但粗略地说，这三者都有类似的趋势。我们的算法和 Sensitivity 非常接近，并且从未超过基线的两倍（通常要好得多）。



(注：KDD 数据集上 Sensitivity 算法在相对大小为 $5 \cdot 10^{-3}$ 时的相对代价峰值是由离群值引起的。)
图 2：四个数据集上相对成本与核心集大小的关系图。阴影区域表示标准误差。核心集大小小于 k 时，没有相对大小值的数据。

- 区别我们（及 Sensitivity）与 Lightweight 的主要差异在于第四列，即合成高斯数据集。对于所有 k 设置，随着核心集大小的增加，Lightweight 显示出的代价下降极小，并且比我们和 Sensitivity（以及基线）差 2.717 倍。这是预期的，因为我们构建的合成高斯数据集使得 Lightweight 具有任意高的代价。由于其乘性近似保证，我们的算法不会遭受这种退化。从这个意义上说，我们的算法更“鲁棒”，并且在所有数据集上都实现了最坏情况下的乘性近似保证。

运行时间 (Running time)： 在表 1（的第一张表）中，我们显示了随着 k 增加，核心集构建算法的运行时间。请注意，随着 k 的增加，我们的算法和 Lightweight 相对于 Sensitivity 的相对加速比在增加。这是因为我们的算法和 Lightweight 的运行时间不随 k 增长。相比之下，Sensitivity 的运行时间随 k 线性增长。总之，对于较大的 k，我们的核心集构建比 Sensitivity 快 33-192 倍。此外，根据数据集的不同，我们的算法比 Lightweight 慢约 35 倍。我们的分析也表明了这一点；两种算法都对数据集进行初始遍历，使用 $O(nd)$ 时间，但我们的算法使用额外的 $O(n \log n)$ 时间来处理.

3.2 直接与 k-means++ 比较

实验设置： 本实验将我们的算法与 k-means++ 作为独立的聚类算法进行比较，而不是作为核心集流程的一部分。我们实现了算法的三个变体。每个变体的区别在于我们如何采样随机一维投影。第一种是投影到**标准高斯向量**（零均值和单位协方差）上的一维投影。这种方法有导致“重要”特征（即具有高方差的特征）坍缩（collapsing）的风险。为了缓解这个问题，我们实现了两个**数据依赖型（data-dependent）**的变体，分别使用数据的方差和协方差。
具体来说，在“**方差（variance）**”变体中，我们使用对角协方差矩阵，其中对角线上的每个条目被设置为数据集沿相应特征的经验方差。在“**协方差（covariance）**”变体中，我们使用数据集的经验协方差矩阵。这些变体旨在沿能够比均匀随机采样向量捕捉更多数据方差的向量进行投影。直观地说，由有偏变体（biased variants）采样的向量与数据集的第一主成分（first principal component）相关性更强。对于我们的每个算法，我们评估输出中心集 C 的 k-means 代价：将点分配给最近中心时的代价（定义 A.1 中的 $\mathrm{cost}_2(X, C)$）；使用我们算法生成的分配时的代价（方程 (1) 中定义的 $\mathrm{cost}_2(X, C, \sigma)$）。

我们针对 $\{10, 25, 50, 100, 250, 500, 1000, 2500, 5000\}$ 中的每个 k 和 $z = 2$ 评估了算法，用于求解基于 ℓ_2 度量的 k-means。在评估分配代价时，我们运行了每一个算法 100 次，对于每个 k，在计算最近邻分配时运行 5 次，我们报告解的平均代价和平均运行时间。由于方差较低且运行时间长得多，k-means++ 运行了 5 次。

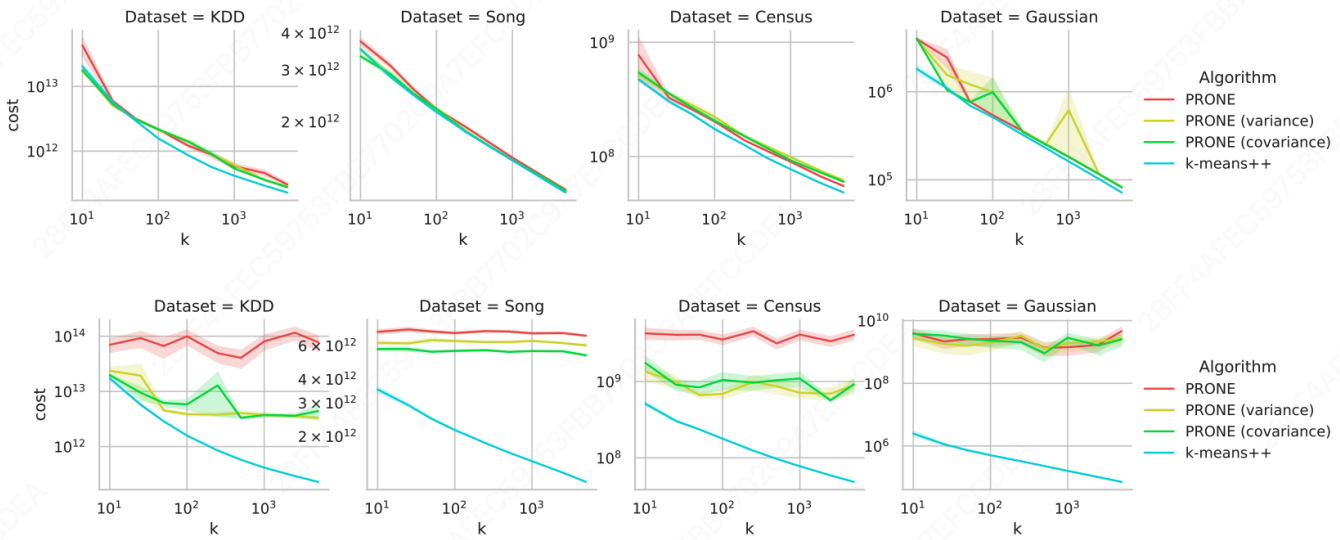


图 3： 我们所有变体与 k-means++ 相比的聚类代价。第一行显示 k-means 代价，第二行显示我们算法产生的分配代价。

Dataset	k Algorithm	10	100	1000
Census	Lightweight	7.3	69.4	670.2
	PRONE coreset	1.5	14.1	136.3
	Sensitivity	1.0	1.0	1.0
Song	Lightweight	8.9	87.2	875.3
	PRONE coreset	2.1	19.9	187.9
	Sensitivity	1.0	1.0	1.0
KDD	Lightweight	6.4	63.0	642.8
	PRONE coreset	2.1	19.6	192.6
	Sensitivity	1.0	1.0	1.0
Gaussian	Lightweight	2.4	17.6	174.8
	PRONE coreset	0.5	3.8	33.7
	Sensitivity	1.0	1.0	1.0

Dataset	k Algorithm	50	500	5000
Census	PRONE	7.5	73.2	662.5
	PRONE (variance)	2.2	22.2	214.7
	PRONE (covariance)	1.1	10.7	117.4
	k-means++	1.0	1.0	1.0
Song	PRONE	9.7	95.5	837.5
	PRONE (variance)	2.3	23.1	217.2
	PRONE (covariance)	0.8	8.2	82.4
	k-means++	1.0	1.0	1.0
KDD	PRONE	6.9	68.3	727.5
	PRONE (variance)	3.1	32.0	312.4
	PRONE (covariance)	1.3	12.9	128.4
	k-means++	1.0	1.0	1.0
Gaussian	PRONE	1.9	18.3	165.9
	PRONE (variance)	2.0	17.7	162.9
	PRONE (covariance)	1.7	16.1	152.6
	k-means++	1.0	1.0	1.0

表 1： 构建核心集时相对于敏感度采样的平均加速比（第一张表），以及作为独立聚类算法相对于 k-means++ 的平均加速比（第二张表）。包含完整参数范围的表格可在附录中找到。

对于每个 k 值，算法运行 100 次；在计算最近邻分配时运行 5 次。我们报告了解决方案的平均成本和平均运行时间。由于方差较低且运行时间更长，kmeans++ 算法运行了 5 次。

直接与 k-means++ 比较的结果：

代价 (Cost)： 图 3（顶部）显示了我们算法找到的中心与 k-means++ 找到的中心的代价对比，这是在计算点到算法计算出的中心的**最优分配**之后（计算此分配耗时 $O(ndk)$）。即，我们比较定义 A.1 中的 $\mathrm{cost}_2(X, C)$ 值。总之，我们算法的三个变体的 k-means 代价大致相同，并且与 k-means++ 非常接近。在高斯数据集上，有偏算法的一次运行未能从原点处的簇中选出中心，导致了较高的“离群值”代价以及图中相应的尖峰。

我们还比较了我们算法计算出的分配的 k-means 代价（这样我们的算法仅耗时 $O(nd + n \log n)$ 而非 $O(ndk)$）与 k-means++ 的代价（图 3 底部行）。即，我们比较方程 (1) 中定义的 $\mathrm{cost}_2(X, C, \sigma)$ 值。我们算法的聚类代价高于 k-means++。这是我们理论结果预期的结果；回顾定理 2.2 给出了 $\mathrm{poly}(k)$-近似，而 k-means++ 是 $O(\log k)$。在真实数据集上，对于我们的无偏变体，其代价比 k-means++ 差一个数量级（$k = 10$）到两个数量级（$k = 5000$）；对于我们的有偏和协方差变体，其代价比 k-means++ 差 2 倍（$k = 10$）到一个数量级（$k = 5000$）。

运行时间 (Running time)： 表 1 显示了我们算法相对于 k-means++ 的相对运行时间，假设不计算最近中心分配。我们的算法设计为运行时间独立于 k，因此我们可以从表 1（及图 1 中的第二张表）看到，我们所有的变体都提供了显著的加速。

- 随着 k 增加，我们算法的运行时间几乎保持恒定，而 k-means++ 的运行时间随 k 线性增长。具体来说，对于 $k = 25$，即使是我们最慢的变体也具有与 k-means++ 大致相同的运行时间，而对于 $k = 5000$，它至少快 82 倍，我们最快的版本比 k-means++ 快高达 837 倍。

- 这两个变体可以将选定中心的质量提高多达一个数量级，但它们也明显更慢。“方差”和“协方差”变体比标准变体慢（分别慢 2-4 倍和高达 10 倍），并且随着维度 d 的增加，它们也会变慢。我们认为这些方法可以进一步加速，因为 blaze 库的方差计算例程对于我们的用例似乎效率低下。

4. 结论与局限性

总而言之，我们提出了一种简单的算法，在运行时间和近似比之间提供了一种新的权衡。我们的算法在期望时间 $O(\mathrm{nnz}(X) + n \log n)$ 内运行，产生 $\mathrm{poly}(k)$-近似；通过额外的 $\mathrm{poly}(kd) \cdot \log n$ 时间，我们将近似比改进为 $O(\log k)$。后一个界限与 k-means++ 相匹配，但提供了显著的加速。

在构建核心集的流程中，我们的实验表明，生成的核心集质量（当使用我们的算法作为初始近似时）优于敏感度采样算法。它比轻量级核心集算法慢，但更具“鲁棒性”，因为它独立于数据集的直径。它不存在加性误差随数据集直径线性增长的缺陷，该缺陷可能会任意增加轻量级核心集算法的代价。当为我们算法返回的中心计算最优分配时，其代价大致与 k-means++ 的代价相匹配。当直接使用我们算法的一个变体产生的分配时，其代价比 k-means++ 差 2 到 10 倍，但速度快高达 300 倍。

我们的实验和运行时间分析表明，我们的算法非常高效。然而，我们的算法实现的聚类质量有时不如其他较慢的算法好。我们表明，当我们使用我们的算法构建核心集时，这种局限性微不足道。一个有趣的开放问题是，在保持像我们一样高效（即运行时间为 $O(nd + n \log n)$）的同时，算法能达到的最佳聚类质量（例如，就近似比而言）是多少。另一个有趣的问题是，是否存在其他将数据集投影到 $O(1)$ 维空间的方法，从而产生具有改进近似保证且运行时间快于 $O(ndk)$ 的算法。

致谢与资金公布

Moses Charikar 获得了 Simons Investigator 奖的支持。

Lunjia Hu 获得了 Moses Charikar 和 Omer Reingold 的 Simons Investigators 奖、Omer Reingold 的 NSF 奖项 IIS-1908774 以及 Simons 基金会算法公平性理论合作项目的支持。这项工作的一部分是在 Erik Waingarten 于斯坦福大学做博士后期间完成的，由 NSF 博士后奖学金和 Moses Charikar 的 Simons Investigator 奖支持。

本项目获得了欧洲研究委员会 (ERC) 在欧盟地平线 2020 研究与创新计划下的资助（资助协议号 101019564，“现代全动态数据结构的设计 (MoDynStruct)”），以及奥地利科学基金 (FWF) 项目 Z 422-N，项目“静态和动态分层图分解”，I 5982-N，以及项目“反应式网络层的快速算法 (ReactNet)”，P 33775-N，并获得了 netidee SCIENCE 基金会 2020-2024 年的额外资助。

引用

[1] Pankaj K Agarwal, Sariel Har-Peled, Kasturi R Varadarajan, et al. Geometric approximation via coresets. *Combinatorial and computational geometry*, 52(1):1–30, 2005.

[2] Sara Ahmadian, Ashkan Norouzi-Fard, Ola Svensson, and Justin Ward. Better guarantees for k-means and Euclidean k-median by primal-dual algorithms. In *2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 61–72, 2017.

[3] Daniel Aloise, Amit Deshpande, Pierre Hansen, and Preyas Popat. NP-hardness of Euclidean sum-of-squares clustering. *Machine Learning*, 75(2):245–248, 2009.

[4] Alexandr Andoni, Piotr Indyk, and Ilya Razenshteyn. Approximate nearest neighbor search in high dimensions. In *Proceedings of the International Congress of Mathematicians: Rio de Janeiro 2018*, pages 3287–3318. World Scientific, 2018.

[5] Sanjeev Arora, Prabhakar Raghavan, and Satish Rao. Approximation schemes for Euclidean k-medians and related problems. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing (STOC '98)*, page 106–113, New York, NY, USA, 1998. Association for Computing Machinery.

[6] David Arthur and Sergei Vassilvitskii. k-means++: the advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035. ACM, New York, 2007.

[7] Olivier Bachem, Mario Lucic, Hamed Hassani, and Andreas Krause. Fast and provably good seedings for k-means. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016.

[8] Olivier Bachem, Mario Lucic, S. Hamed Hassani, and Andreas Krause. Approximate k-means++ in sublinear time. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1), Feb. 2016.

[9] Olivier Bachem, Mario Lucic, and Andreas Krause. Practical coreset constructions for machine learning. *arXiv preprint arXiv:1703.06476*, 2017.

[10] Olivier Bachem, Mario Lucic, and Andreas Krause. Scalable k-means clustering via lightweight coresets. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '18)*, page 1119–1127, New York, NY, USA, 2018. Association for Computing Machinery.

[11] Luca Becchetti, Marc Bury, Vincent Cohen-Addad, Fabrizio Grandoni, and Chris Schwiegelshohn. Oblivious dimension reduction for k-means: Beyond subspaces and the johnson-lindenstrauss lemma. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing (STOC 2019)*, page 1039–1050, New York, NY, USA, 2019. Association for Computing Machinery.

[12] Thierry Bertin-Mahieux, Daniel P.W. Ellis, Brian Whitman, and Paul Lamere. The million song dataset. In *Proceedings of the 12th International Conference on Music Information Retrieval (ISMIR 2011)*, 2011.

[13] Thomas Bottesch, Thomas Bühler, and Markus Kächele. Speeding up k-means by approximating Euclidean distances via block vectors. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 2578–2586, New York, New York, USA, 20–22 Jun 2016. PMLR.

[14] Christos Boutsidis, Anastasios Zouzias, Michael W Mahoney, and Petros Drineas. Randomized dimensionality reduction for k-means clustering. *IEEE Transactions on Information Theory*, 61(2):1045–1062, 2014.

[15] Vladimir Braverman, Dan Feldman, and Harry Lang. New frameworks for offline and streaming coreset constructions. *CoRR*, abs/1612.00889, 2016.

[16] James Briggs. Faiss: The missing manual. https://www.pinecone.io/learn/faiss/.

[17] Jarosław Byrka, Thomas Pensyl, Bartosz Rybicki, Aravind Srinivasan, and Khoa Trinh. An improved approximation for k-median, and positive correlation in budgeted optimization. In *Proceedings of the Twenty-Sixth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA '15)*, page 737–756, USA, 2015. Society for Industrial and Applied Mathematics.

[18] Marco Capó, Aritz Pérez, and Jose A Lozano. An efficient k-means clustering algorithm for massive data. *arXiv preprint arXiv:1801.02949*, 2018.

[19] Ke Chen. On coresets for k-median and k-means clustering in metric and Euclidean spaces and their applications. *SIAM Journal on Computing*, 39(3):923–947, 2009.

[20] Davin Choo, Christoph Grunau, Julian Portmann, and Vaclav Rozhon. k-means++: few more steps yield constant approximation. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 1909–1917. PMLR, 13–18 Jul 2020.

[21] Michael B. Cohen, Sam Elder, Cameron Musco, Christopher Musco, and Madalina Persu. Dimensionality reduction for k-means clustering and low rank approximation. In *Proceedings of the Forty-Seventh Annual ACM Symposium on Theory of Computing (STOC '15)*, page 163–172, New York, NY, USA, 2015. Association for Computing Machinery.

[22] Vincent Cohen-Addad. A fast approximation scheme for low-dimensional k-means. In *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA '18)*, page 430–440, USA, 2018. Society for Industrial and Applied Mathematics.

[23] Vincent Cohen-Addad, Andreas Emil Feldmann, and David Saulpic. Near-linear time approximation schemes for clustering in doubling metrics. *J. ACM*, 68(6), oct 2021.

[24] Vincent Cohen-Addad, Philip N. Klein, and Claire Mathieu. Local search yields approximation schemes for k-means and k-median in Euclidean and minor-free metrics. In *2016 IEEE 57th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 353–364, 2016.

[25] Vincent Cohen-Addad, Kasper Green Larsen, David Saulpic, and Chris Schwiegelshohn. Towards optimal lower bounds for k-median and k-means coresets. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*, pages 1038–1051, 2022.

[26] Vincent Cohen-Addad, Silvio Lattanzi, Ashkan Norouzi-Fard, Christian Sohler, and Ola Svensson. Fast and accurate k-means++ via rejection sampling. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 16235–16245. Curran Associates, Inc., 2020.

[27] Vincent Cohen-Addad, David Saulpic, and Chris Schwiegelshohn. A new coreset framework for clustering. In *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*, pages 169–182, 2021.

[28] Ryan R Curtin. A dual-tree algorithm for fast k-means clustering with large k. In *Proceedings of the 2017 SIAM International Conference on Data Mining*, pages 300–308. SIAM, 2017.

[29] Yufei Ding, Yue Zhao, Xipeng Shen, Madanlal Musuvathi, and Todd Mytkowicz. Yinyang k-means: A drop-in replacement of the classic k-means with consistent speedup. In Francis Bach and David Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 579–587, Lille, France, 07–09 Jul 2015. PMLR.

[30] Jonathan Drake. *Faster k-means clustering*. PhD thesis, Baylor University, 2013.

[31] Petros Drineas, Alan Frieze, Ravi Kannan, Santosh Vempala, and Vishwanathan Vinay. Clustering large graphs via the singular value decomposition. *Machine learning*, 56:9–33, 2004.

[32] Dheeru Dua and Casey Graff. UCI machine learning repository, 2017.

[33] Charles Elkan. Using the triangle inequality to accelerate k-means. In *Proceedings of the Twentieth International Conference on Machine Learning (ICML'03)*, page 147–153. AAAI Press, 2003.

[34] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD'96)*, page 226–231. AAAI Press, 1996.

[35] Dan Feldman. Introduction to coresets: an updated survey. *arXiv preprint arXiv:2011.09384*, 2020.

[36] Dan Feldman and Michael Langberg. A unified framework for approximating and clustering data. In *Proceedings of the forty-third annual ACM symposium on Theory of computing*, pages 569–578, 2011.

[37] Dan Feldman, Melanie Schmidt, and Christian Sohler. Turning big data into tiny data: Constant-size coresets for k-means, pca, and projective clustering. *SIAM Journal on Computing*, 49(3):601–657, 2020.

[38] Zachary Friggstad, Mohsen Rezapour, and Mohammad R. Salavatipour. Local search yields a ptas for k-means in doubling metrics. In *2016 IEEE 57th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 365–374, 2016.

[39] Fabrizio Grandoni, Rafail Ostrovsky, Yuval Rabani, Leonard J. Schulman, and Rakesh Venkat. A refined approximation for Euclidean k-means. *Inf. Process. Lett.*, 176(C), jun 2022.

[40] Greg Hamerly. Making k-means even faster. In *Proceedings of the 2010 SIAM international conference on data mining*, pages 130–140. SIAM, 2010.

[41] Sariel Har-Peled and Soham Mazumdar. On coresets for k-means and k-median clustering. In *Proceedings of the Thirty-Sixth Annual ACM Symposium on Theory of Computing (STOC '04)*, page 291–300, New York, NY, USA, 2004. Association for Computing Machinery.

[42] Oktie Hassanzadeh, Fei Chiang, Hyun Chul Lee, and Renée J. Miller. Framework for evaluating clustering algorithms in duplicate detection. *Proc. VLDB Endow.*, 2(1):1282–1293, aug 2009.

[43] Jennifer Jang and Heinrich Jiang. Dbscan++: Towards fast and scalable density clustering. In *International conference on machine learning*, pages 3019–3029. PMLR, 2019.

[44] Herve Jegou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence*, 33(1):117–128, 2010.

[45] T. Kanungo, D.M. Mount, N.S. Netanyahu, C.D. Piatko, R. Silverman, and A.Y. Wu. An efficient k-means clustering algorithm: analysis and implementation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 24(7):881–892, 2002.

[46] Tapas Kanungo, David M. Mount, Nathan S. Netanyahu, Christine Piatko, Ruth Silverman, and Angela Y. Wu. The analysis of a simple k-means clustering algorithm. In *Proceedings of the Sixteenth Annual Symposium on Computational Geometry (SCG '00)*, page 100–109, New York, NY, USA, 2000. Association for Computing Machinery.

[47] 2004 KDD Cup. Protein homology dataset. available at https://osmot.cs.cornell.edu/kddcup/datasets.html, 2004.

[48] Stavros G. Kolliopoulos and Satish Rao. A nearly linear-time approximation scheme for the Euclidean k-median problem. In Jaroslav Nešetřil, editor, *Algorithms - ESA' 99*, pages 378–389, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.

[49] Silvio Lattanzi and Christian Sohler. A better k-means++ algorithm via local search. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 3662–3671. PMLR, 09–15 Jun 2019.

[50] Shi Li and Ola Svensson. Approximating k-median via pseudo-approximation. In *Proceedings of the Forty-Fifth Annual ACM Symposium on Theory of Computing (STOC '13)*, page 901–910, New York, NY, USA, 2013. Association for Computing Machinery.

[51] Weiwei Liu, Xiaobo Shen, and Ivor Tsang. Sparse embedded k-means clustering. *Advances in neural information processing systems*, 30, 2017.

[52] Stuart P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

[53] Konstantin Makarychev, Yury Makarychev, and Ilya Razenshteyn. Performance of johnson-lindenstrauss transform for k-means and k-medians clustering. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing (STOC 2019)*, page 1027–1038, New York, NY, USA, 2019. Association for Computing Machinery.

[54] Andrew W. Moore. The anchors hierarchy: Using the triangle inequality to survive high dimensional data. In *Proceedings of the Sixteenth Conference on Uncertainty in Artificial Intelligence (UAI'00)*, page 397–405, San Francisco, CA, USA, 2000. Morgan Kaufmann Publishers Inc.

[55] James Newling and Francois Fleuret. Fast k-means with accurate bounds. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 936–944, New York, New York, USA, 20–22 Jun 2016. PMLR.

[56] Raymond T. Ng and Jiawei Han. Clarans: A method for clustering objects for spatial data mining. *IEEE transactions on knowledge and data engineering*, 14(5):1003–1016, 2002.

[57] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[58] Dan Pelleg and Andrew Moore. Accelerating exact k-means algorithms with geometric reasoning. In *Proceedings of the Fifth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '99)*, page 277–281, New York, NY, USA, 1999. Association for Computing Machinery.

[59] James Philbin. *Scalable object retrieval in very large image collections*. PhD thesis, Oxford University, 2010.

[60] James Philbin, Ondrej Chum, Michael Isard, Josef Sivic, and Andrew Zisserman. Object retrieval with large vocabularies and fast spatial matching. In *2007 IEEE Conference on Computer Vision and Pattern Recognition*, pages 1–8, 2007.

[61] Feng Qian, Abhinav Pathak, Yu Charlie Hu, Zhuoqing Morley Mao, and Yinglian Xie. A case for unsupervised-learning-based spam filtering. *SIGMETRICS Perform. Eval. Rev.*, 38(1):367–368, jun 2010.

[62] M Ali Rostami, Alieh Saeedi, Eric Peukert, and Erhard Rahm. Interactive visualization of large similarity graphs and entity resolution clusters. In *EDBT*, pages 690–693, 2018.

[63] Erich Schubert and Peter J Rousseeuw. Faster k-medoids clustering: improving the pam, clara, and clarans algorithms. In *Similarity Search and Applications: 12th International Conference, SISAP 2019*, Newark, NJ, USA, October 24, 2019, Proceedings 12, pages 171–187. Springer, 2019.

[64] D. Sculley. Web-scale k-means clustering. In *Proceedings of the 19th International Conference on World Wide Web (WWW '10)*, page 1177–1178, New York, NY, USA, 2010. Association for Computing Machinery.

[65] Mina Sheikhalishahi, Andrea Saracino, Mohamed Mejri, Nadia Tawbi, and Fabio Martinelli. Fast and effective clustering of spam emails based on structural similarity. In Joaquin Garcia-Alfaro, Evangelos Kranakis, and Guillaume Bonfante, editors, *Foundations and Practice of Security*, pages 195–211, Cham, 2016. Springer International Publishing.

[66] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamny, Gopal Srinivasa, et al. Results of the neurips'21 challenge on billion-scale approximate nearest neighbor search. In *NeurIPS 2021 Competitions and Demonstrations Track*, pages 177–189. PMLR, 2022.

[67] Christian Sohler and David P Woodruff. Strong coresets for k-median and subspace approximation: Goodbye dimension. In *2018 IEEE 59th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 802–813. IEEE, 2018.

[68] Kunal Talwar. Bypassing the embedding: Algorithms for low dimensional metrics. In *Proceedings of the Thirty-Sixth Annual ACM Symposium on Theory of Computing (STOC '04)*, page 281–290, New York, NY, USA, 2004. Association for Computing Machinery.

[69] Mo Tiwari, Martin J Zhang, James Mayclin, Sebastian Thrun, Chris Piech, and Ilan Shomorony. Banditpam: Almost linear time k-medoids clustering via multi-armed bandits. *Advances in Neural Information Processing Systems*, 33:10211–10222, 2020.

[70] Erik Waingarten. Notes for algorithms for big data: Clustering. https://drive.google.com/file/d/1T5YYGrA3kdi4QGvF3cfoOZvXMDaRPw/view.

[71] Jing Wang, Jingdong Wang, Qifa Ke, Gang Zeng, and Shipeng Li. Fast approximate k-means via cluster closures. In *2012 IEEE Conference on Computer Vision and Pattern Recognition*, pages 3037–3044, 2012.

A. 问题设置与背景 (Problem Setup and Background)

在这项工作中，我们始终考虑高维向量数据集 $X = \{x_1, \ldots, x_n\} \subset \mathbb{R}^d$，并且我们将使用欧几里得 ($\ell_2$) 距离来度量它们的距离。下面，我们定义 (k, z)-聚类。这个问题引入了一个参数 $z \geq 1$，用于衡量对离群值的敏感度（随着 z 增大，聚类对距离簇中心最远的点的敏感度增加）。k-means 的情况对应于 $z = 2$，但 z 的其他值捕捉了其他著名的聚类目标，如 k-median（$z = 1$ 的情况）。

定义 A.1 ((k, z)-聚类)： 考虑数据集 $X = \{x_1, \ldots, x_n\} \subset \mathbb{R}^d$，期望的簇数量 $k \in \mathbb{N}$，以及参数 $z \geq 1$。对于一组 k 个中心 $C = \{c_1, \ldots, c_k\} \subset \mathbb{R}^d$，令 $\mathrm{cost}_z(X, C)$ 表示使用中心集 C 对 X 进行聚类的代价，即

$$\mathrm{cost}_z(X, C) = \sum_{i=1}^{n} \min_{j \in [k]} \|x_i - c_j\|_2^z.$$

我们令 $\mathrm{opt}_{k,z}(X)$ 表示在所有 $C=\{c_1,\ldots,c_k\}\subset\mathbb{R}^d$ 选择下的最优代价：

$$\mathrm{opt}_{k,z}(X)=\inf_{\substack{C\subset\mathbb{R}^d\\|C|\le k}}\mathrm{cost}_z(X,C).$$

一个 (k,z)-聚类算法具有以下规格。算法接收数据集 $X=\{x_1,\ldots,x_n\}\subset\mathbb{R}^d$ 以及两个参数 $k\in\mathbb{N}$ 和 $z\ge 1$ 作为输入。执行后，算法应输出一组 k 个中心 $C=\{c_1,\ldots,c_k\}\subset\mathbb{R}^d$ 以及一个分配 $\sigma:[n]\to[k]$，将每个点 x_i 映射到中心 $c_{\sigma(i)}$。

我们使用解 (C,σ) 的 (k,z)-聚类代价 $\mathrm{cost}_z(X,C,\sigma)$ 与最优代价 $\mathrm{opt}_{k,z}(X)$ 之间的比率来衡量解的质量，其中

$$\mathrm{cost}_z(X,C,\sigma):=\sum_{i=1}^n\|x_i-c_{\sigma(i)}\|_2^z. \quad (1)$$

对于任意 $D>1$，一个产生 (k,z)-聚类 D-近似的算法应保证 $\mathrm{cost}_z(X,C,\sigma)$ 最多为 $D\cdot\mathrm{opt}_{k,z}(X)$。对于随机算法，该保证应以大概率（称为成功概率）对任意输入数据集成立。

A.1 k-Means++ 初始化 (k-Means++ Seeding)

k-means++ 初始化算法是 [6] 中引入的一个经过充分研究的算法，它是我们算法的重要组成部分。下面，我们针对一般的 $z\ge 1$（不一定是 $z=2$）对其进行描述。

定义 A.2 (针对任意 $z\ge 1$ 的 k-means++ 初始化)： 给定 n 个数据点 $x_1,\ldots,x_n\in\mathbb{R}^d$，$k$-means++ 初始化算法通过以下过程产生一组 k 个中心 $x_{\ell_1},\ldots,x_{\ell_k}$：

1. 从 $[n]$ 中均匀随机地选择 ℓ_1。
2. 对于 $t=2,\ldots,k$，按如下方式采样 ℓ_t。对于每个 $i\in[n]$，令 p_i 表示 x_i 到 $x_{\ell_1},\ldots,x_{\ell_{t-1}}$ 中最近点的欧几里得距离。从 $[n]$ 中采样 ℓ_t，使得对于每个 $i\in[n]$，概率 $\Pr[\ell_t=i]$ 与 p_i^z 成正比。即，

$$\Pr[\ell_t=i]=\frac{p_i^z}{\sum_{i'\in[n]}p_{i'}^z}.$$

   在 k-means 的背景下（即当 $z=2$ 时），该分布被称为 D^2 分布。
3. 输出 $x_{\ell_1},\ldots,x_{\ell_k}$。

在上述描述中，k-means++ 的第 2 步需要为每个数据点 x_i 维护其到当前迭代之前所选中心中最近中心的欧几里得距离 p_i。这一步是通过在第 2 步的 $k-1$ 次迭代中每次都遍历整个数据集来实现的，导致 $O(ndk)$ 的运行时间。

定理 A.1 ([6], [70] 中的定理 3)： 对于 $x_1,\ldots,x_n\in\mathbb{R}^d$，令 $X=\{x_1,\ldots,x_n\}$ 为 k-means++ 初始化算法的输入。对于 k-means++ 初始化算法的输出 $x_{\ell_1},\ldots,x_{\ell_k}$，定义 $C=\{x_{\ell_1},\ldots,x_{\ell_k}\}$。则

$$\mathbb{E}[\mathrm{cost}_z(X,C)]=O(2^{2z}\log k)\cdot\mathrm{opt}_{k,z}(X).$$

A.2 基于敏感度采样的核心集 (Coresets via Sensitivity Sampling)

我们算法的应用之一是构建 (k,z)-聚类的核心集。我们给出了正式定义，并描述了构建核心集的主要技术。

定义 A.3： 给定数据集 $X=\{x_1,\ldots,x_n\}\subset\mathbb{R}^d$，以及参数 $k\in\mathbb{N},z\ge 1$ 和 $\epsilon>0$，(k,z)-聚类的一个（强）ϵ-核心集由点集 $Y\subset\mathbb{R}^d$ 和权重函数 $w:Y\to\mathbb{R}_{\ge 0}$ 指定，使得对于每个集合 $C=\{c_1,\ldots,c_k\}\subset\mathbb{R}^d$，

$$(1-\epsilon)\cdot\mathrm{cost}_z(X,C)\le\sum_{y\in Y}w(y)\cdot\min_{j\in[k]}\|y-c_j\|_2^z\le(1+\epsilon)\cdot\mathrm{cost}_z(X,C).$$

核心集是通过“敏感度采样（sensitivity sampling）”构建的，这是一种技术，给定数据集 X 的一个近似聚类，它产生一个概率分布，使得从该分布中采样足够多的点会产生一个核心集。

定义 A.4 (敏感度采样)： 考虑数据集 $X=\{x_1,\ldots,x_n\}\subset\mathbb{R}^d$，以及参数 $k\in\mathbb{N},z\ge 1$。对于中心集 $C=\{c_1,\ldots,c_k\}\subset\mathbb{R}^d$ 和分配 $\sigma:[n]\to[k]$，令 $X_j=\{x_i:\sigma(i)=j\}$。我们令 $\mathcal{D}$ 为支持在 X 上的分布，其中

$$\Pr_{x\sim\mathcal{D}}[x=x_i]\propto\frac{\|x_i-c_{\sigma(i)}\|_2^z}{\sum_{j=1}^n\|x_j-c_{\sigma(j)}\|_2^z}+\frac{1}{|X_{\sigma(i)}|}.$$

我们将使用的主要定理如下，它表明给定一个给出 (k,z)-聚类 α-近似的中心集和分配，可以从上述定义的分布 $\mathcal{D}$ 中采样以高概率生成核心集。

定理 A.2 ([15])： 对于任意数据集 $X=\{x_1,\dots,x_n\}\subset\mathbb{R}^d$ 和任意参数 $k\in\mathbb{N}$ 和 $z\geq 1$，假设 $C=\{c_1,\dots,c_k\}\subset\mathbb{R}^d$ 和 $\sigma:[n]\to[k]$ 是 (k,z)-聚类的 α-近似，即

$$\sum_{i=1}^{n}\|x_i-c_{\sigma(i)}\|_2^z\leq\alpha\cdot\mathrm{opt}_{k,z}(X).$$

令 $\mathcal{D}$ 表示定义 A.4 中指定的分布，以下情况以高概率发生。

- 我们令 $y_1,\dots,y_s$ 表示从 $\mathcal{D}$ 中独立采样的样本，并且 $w(y_i)$ 是 y_i 根据 $\mathcal{D}$ 被采样的概率的倒数。我们设置 $s\geq\mathrm{poly}(kd\cdot\alpha\cdot 2^{z/\epsilon})$。
- 带有权重 w 的集合 $Y=\{y_1,\dots,y_s\}$ 是 (k,z)-聚类的 ϵ-核心集。

A.3 一个简单的引理 (A Simple Lemma)

我们将反复使用以下简单的引理。

引理 A.3： 令 $a,b\in\mathbb{R}_{\geq 0}$ 为任意两个数且 $z\geq 1$。则 $(a+b)^z\leq 2^{z-1}a^z+2^{z-1}b^z$。

证明： 函数 $\phi(t)=t^z$ 对于 $z\geq 1$ 是凸函数，因此 Jensen 不等式意味着 $\phi((a+b)/2)\leq(1/2)\phi(a)+(1/2)\phi(b)$。

B. 基于一维投影的近似保证 (Approximation Guarantees from One-Dimensional Projections)

在本节中，我们通过分析算法中的随机一维投影步骤来证明定理 2.2（更确切地说，是定理 2.2 对任意 $z\geq 1$ 的推广）。为了引入一些符号，令 $X=\{x_1,\dots,x_n\}\subset\mathbb{R}^d$ 为一组点，对于 X 划分为 k 个集合的划分 $(Y_1,\dots,Y_k)$，我们令 X 在划分 $(Y_1,\dots,Y_k)$ 下的 (k,z)-聚类代价为

$$\mathrm{cost}_z(Y_1,\dots,Y_k)=\sum_{i=1}^{k}\min_{c\in\mathbb{R}^d}\sum_{x\in Y_i}\|x-c\|_2^z\quad(2)$$

并将被选为最小值的 k 个点 c 称为实现 $(Y_1,\dots,Y_k)$ 的 (k,z)-聚类代价的一组中心。我们注意到 (2) 是 (k,z)-聚类的一个代价函数，但它与定义 A.1 不同。在定义 A.1 中，重点在于 k 个中心的集合 $C=\{c_1,\dots,c_k\}$，而 X 的诱导聚类集（即通过将点分配给最近中心给出的划分 $(Y_1,\dots,Y_k)$）仅由中心集隐式指定。另一方面，(2) 强调聚类 $(Y_1,\dots,Y_k)$，以及由 $(Y_1,\dots,Y_k)$ 隐式指定的 k 个中心集。最优中心集和最优聚类将实现相同的代价；然而，我们的证明将主要考虑聚类 $(Y_1,\dots,Y_k)$ 作为优化对象。稍后，我们将采样一个（随机）降维映射 $\Pi:\mathbb{R}^d\to\mathbb{R}^t$ 并寻求 $t=1$ 时的界限。我们将用 $\mathrm{cost}_z(\Pi(Y_1),\dots,\Pi(Y_k))$ 表示对划分 $Y_1,\dots,Y_k$ 应用降维映射 Π 后聚类点的代价。即，我们写

$$\mathrm{cost}_z(\Pi(Y_1),\dots,\Pi(Y_k))=\sum_{i=1}^{k}\min_{c\in\mathbb{R}^t}\sum_{x\in Y_i}\|\Pi(x)-c\|_2^z.$$

定义 B.1： 对于点集 $X=\{x_1,\dots,x_n\}\subset\mathbb{R}^d$，我们用 $X_1^*,\dots,X_k^*$ 表示 X 具有最小 (k,z)-聚类代价的划分，并用 $C^*=\{c_1^*,\dots,c_k^*\}\subset\mathbb{R}^d$ 表示实现 $X_1^*,\dots,X_k^*$ 的 (k,z)-聚类代价的一组 k 个中心，即满足以下条件的中心集：

$$\mathrm{cost}_z(X_1^*,\dots,X_k^*)=\sum_{i=1}^{k}\sum_{x\in X_i^*}\|x-c_i^*\|_2^z.$$

通过稍微滥用符号，我们也令 $c^*:X\to C^*$ 为将 X 的每个点发送到其对应中心的映射（即，如果 $x\in X_i^*$，则 $c^*(x)$ 是点 c_i^*）。

我们证明以下引理，它将引理 2.5 从 k-means 推广到 (k,z)-聚类（回顾 k-means 对应于 $z=2$ 的情况）。

引理 B.1 ((k,z)-聚类上一维投影的影响)： 对于 $n,d,k\in\mathbb{N}$ 和 $z\geq 1$，令 $X=\{x_1,\dots,x_n\}\subset\mathbb{R}^d$ 为任意数据集。我们考虑通过采样 $g\sim\mathcal{N}(0,I_d)$ 并设置 $\Pi(x)=\langle x,g\rangle$ 给出的（随机）线性映射 $\Pi:\mathbb{R}^d\to\mathbb{R}$。
在 g 的选择上，以下情况以至少 0.9 的概率发生：

- 我们考虑投影数据集 $X'=\{x_1',\dots,x_n'\}\subset\mathbb{R}$ 由 $x_i'=\Pi(x_i)$ 给出，并且
- 对于任意 $r\geq 1$，我们令 $(Y_1,\dots,Y_k)$ 表示 X 的任意划分，满足

$$\mathrm{cost}_z(\Pi(Y_1),\dots,\Pi(Y_k))\leq r\cdot\min_{c_1,\dots,c_k\in\mathbb{R}}\sum_{i=1}^{n}\min_{j\in[k]}|x_i'-c_j|^z.$$

则，

$$\mathrm{cost}_z(Y_1,\ldots,Y_k) \le 2^{O(z)} \cdot k^{2z} \cdot r \cdot \mathrm{opt}_{k,z}(X).$$

通过设置 $z=2$，我们获得了引理 2.5 中所需的界限。我们可以立即看到，从引理 B.1 以及定理 A.1 中 k-means++（或者更确切地说是其对 $z \ge 1$ 的推广）的近似保证，我们获得了所需的近似保证。下面，我们陈述定理 2.2 对所有 $z \ge 1$ 的推广，并在假设引理 B.1 成立的情况下给出其证明。

定理 B.2 (定理 2.2 对 $z \ge 1$ 的推广)： 对于 $n,d,k \in \mathbb{N}$ 和 $z \ge 1$，令 $X=\{x_1,\ldots,x_n\} \subset \mathbb{R}^d$ 为任意数据集。我们考虑我们算法 PRONE 的以下推广：

1. 采样一个随机高斯向量 $g \sim \mathcal{N}(0, I_d)$ 并考虑由 $x'_i = \Pi(x_i)$ 给出的投影 $X' = \{x'_1,\ldots,x'_n\}$，其中 $\Pi(x) = \langle x, g\rangle \in \mathbb{R}$。
2. 在数据集 $X' \subset \mathbb{R}$ 上执行定义 A.2 中针对 $z \ge 1$ 的 k-means++ 初始化策略（的推广），令 $x'_{j_1},\ldots,x'_{j_k} \in \mathbb{R}$ 表示中心，$(Y_1,\ldots,Y_k)$ 表示指定找到的 k 个簇的 X 的划分。
3. 输出聚类 $(Y_1,\ldots,Y_k)$，以及中心集 $c_1,\ldots,c_k \in \mathbb{R}^d$，其中 $c_\ell = \mathbb{E}_{x\sim Y_\ell}[x] \in \mathbb{R}^d$。

则，在算法执行的随机性上，以至少 0.8 的概率，

$$\mathrm{cost}_z(Y_1,\ldots,Y_k) \le \sum_{\ell=1}^{k}\sum_{x\in Y_\ell} \|x-c_\ell\|_2^z \le 2^{O(z)} \cdot k^{2z} \cdot \log k \cdot \mathrm{opt}_{k,z}(X).$$

假设引理 B.1 成立证明定理 B.2： 我们考虑（在算法执行的随机性上）以下情况：

1. 引理 B.1 的结论对投影数据集 X' 成立（根据引理 B.1，这以至少 0.9 的概率发生）。
2. 在 X' 上执行推广的 k-means++ 初始化策略（来自定义 A.2）产生一组中心 $\{x'_{j_1},\ldots,x'_{j_k}\} \subset \mathbb{R}$，其聚类 X' 的代价至多为 $O(2^{2z}\log k) \cdot \mathrm{opt}_{k,z}(X')$（根据马尔可夫不等式，这也以 0.9 的概率发生）。

通过联合界（union bound），两者同时成立的概率至少为 0.8。我们现在使用引理 B.1 来上界聚类 $(Y_1,\ldots,Y_k)$ 的代价。第一个不等式是平凡的；假设我们令 $\hat{c}^*_1,\ldots,\hat{c}^*_\ell \in \mathbb{R}^d$ 为对于每个 $\ell \in [k]$ 最小化下式的中心

$$\min_{\hat{c}_\ell \in \mathbb{R}^d} \sum_{x\in Y_\ell} \|x-\hat{c}_\ell\|_2^z = \sum_{x\in Y_\ell} \|x-\hat{c}^*_\ell\|_2^z.$$

那么，我们平凡地有

$$\mathrm{cost}_z(Y_1,\ldots,Y_k) = \sum_{\ell=1}^{k}\sum_{x\in Y_\ell} \|x-\hat{c}^*_\ell\|_2^z \le \sum_{\ell=1}^{k}\sum_{x\in Y_\ell} \|x-c_\ell\|_2^z.$$

此外，我们也可以展示一个相应的上界。对于每个 $\ell \in [k]$，回顾 $c_\ell \in \mathbb{R}^d$ 是 Y_ℓ 的质心，所以我们可以应用三角不等式和引理 A.3

$$\sum_{x\in Y_\ell} \|x-c_\ell\|_2^z \le 2^{z-1}\sum_{x\in Y_\ell} \|x-\hat{c}^*_\ell\|_2^z + 2^{z-1}|Y_\ell| \cdot \|\hat{c}^*_\ell - \mathbb{E}_{x\sim Y_\ell}[x]\|_2^z \le 2^{z-1}\sum_{x\in Y_\ell} \|x-\hat{c}^*_\ell\|_2^z + 2^{z-1}|Y_\ell| \cdot \mathbb{E}_{x\sim Y_\ell}[\|x-\hat{c}^*_\ell\|_2^z],$$

其中第二个不等式是 Jensen 不等式，因为 $\phi(x) = \|\hat{c}^*_\ell - x\|_2^z$ 对于 $z \ge 1$ 是凸函数。
因此，我们有上界

$$\sum_{x\in Y_\ell} \|x-c_\ell\|_2^z \le 2^z \sum_{x\in Y_\ell} \|x-\hat{c}^*_\ell\|_2^z,$$

因此

$$\sum_{\ell=1}^{k}\sum_{x\in Y_\ell} \|x-c_\ell\|_2^z \le 2^z \cdot \mathrm{cost}_z(Y_1,\ldots,Y_k). \quad (3)$$

最后一步涉及使用引理 B.1 的结论关联 $\mathrm{cost}_z(Y_1,\ldots,Y_k)$。注意我们的算法产生的 X' 的聚类 $(Y_1,\ldots,Y_k)$ 是通过令

$$Y_\ell = \{x_i \in X : \forall j' \in [k], |x'_i - x'_{j_\ell}| \le |x'_i - x'_{j'}|^z\}$$

指定的，并且由事件 (2)，我们有 $\mathrm{cost}_z(\Pi(Y_1),\ldots,\Pi(Y_k)) \le O(2^{2z}\log k) \cdot \mathrm{opt}_{k,z}(X')$。由事件 (1)，引理 B.1 意味着 $\mathrm{cost}_z(Y_1,\ldots,Y_k) \le 2^{O(z)} \cdot k^{2z} \cdot O(2^{2z}\log k) \cdot \mathrm{opt}_{k,z}(X)$。结合 (3)，我们获得所需的界限。

B.1 引理 B.1 的证明 (Proof of Lemma B.1)

我们现在转向引理 B.1 的证明，我们的分析将分两步进行。首先，我们假设一个固定的降维映射 $\Pi : \mathbb{R}^d \to \mathbb{R}^t$，它满足关于 Π 的两个几何条件。在这些条件下，我们展示了如何将 $\mathbb{R}^t$ 中映射点的近似聚类“提升”为 $\mathbb{R}^d$ 中原始数据集的近似聚类，代价是削弱近似比。然后，我们证明由 $\Pi(x) = \langle x, g\rangle$ 给出的简单一维投影 $\Pi : \mathbb{R}^d \to \mathbb{R}$（其中 g 从 d 维标准高斯采样）满足我们引理的几何条件。

引理 B.3： 令 $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ 且 $\Pi : \mathbb{R}^d \to \mathbb{R}^t$ 为线性映射。令 $C = \{c_1^*, \dots, c_k^*\} \subset \mathbb{R}^d$ 表示最小化 $\mathrm{cost}_z(X, C)$ 的中心集，$(X_1^*, \dots, X_k^*)$ 表示最优 (k, z)-聚类，并假设对于参数 $D_1, D_2, D_3 \geq 1$，以下条件成立：

- **中心不收缩 (Centers Don't Contract)：** 每个 $i, j \in [k]$ 满足

$$\|c_i^* - c_j^*\|_2 \leq D_1 \cdot \|\Pi(c_i^*) - \Pi(c_j^*)\|_2.$$

- **$(X_1^*, \dots, X_k^*)$ 的代价不增加 (Cost of ... does not Increase)：** 我们有

$$\sum_{i=1}^{k} \sum_{x \in X_i^*} \|\Pi(x) - \Pi(c_i^*)\|_2^z \leq D_2 \cdot \mathrm{cost}_z(X_1^*, \dots, X_k^*).$$

- **近似最优 $(\Pi(Y_1), \dots, \Pi(Y_k))$：** X 的划分 $(Y_1, \dots, Y_k)$ 对于 $\Pi(X)$ 是 D_3-近似最优的，即

$$\mathrm{cost}_z(\Pi(Y_1), \dots, \Pi(Y_k)) \leq D_3 \cdot \min_{c_1, \dots, c_k \in \mathbb{R}^t} \sum_{x \in X} \min_{j \in [k]} \|\Pi(x) - c_j\|_2^z.$$

则，

$$\mathrm{cost}_z(Y_1, \dots, Y_k) \leq \left(2^{3z-1} D_1^z (1 + D_3) + 2^{2z-1}\right) D_2 \cdot \mathrm{cost}_z(X_1^*, \dots, X_k^*).$$

在开始证明引理 B.3 之前，我们展示将点投影到随机高斯向量上能够以至少 $1 - \delta$ 的概率给出上述引理所需的两个保证，其中 $D_1 := O(k^2/\delta)$ 和 $D_2 := 2^{O(z)}/\delta$。我们在下面陈述的第一个引理表明引理 B.3 的第一个条件以高概率满足，第二个引理表明引理 B.3 的第二个条件以高概率满足。

引理 B.4 (中心不收缩)： 令 $\{c_1, \dots, c_k\} \subset \mathbb{R}^d$ 表示任意点集，令 $\Pi : \mathbb{R}^d \to \mathbb{R}$ 为由 $\Pi(x) = \langle x, g\rangle$ 给出的随机映射，其中 $g \sim \mathcal{N}(0, I_d)$ 为随机选择的向量。则以 g 上至少 $1 - \delta$ 的概率，每个 $i, j \in [k]$ 满足

$$\frac{\delta}{k^2} \cdot \|c_i - c_j\|_2 \leq \|\Pi(c_i) - \Pi(c_j)\|_2.$$

引理 B.5 ($(X_1^*, \dots, X_k^*)$ 的代价不增加)： 令 $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$，令 $X_1^*, \dots, X_k^*$ 为 X 的划分，且 $c_1^*, \dots, c_k^*$ 为最小化 X 的 (k, z)-聚类代价的中心。则以至少 $1 - \delta$ 的概率，

$$\sum_{i=1}^{k} \sum_{x \in X_i^*} \|\Pi(x) - \Pi(c_i^*)\|_2^z \leq \frac{2^{O(z)}}{\delta} \cdot \mathrm{cost}_z(X_1^*, \dots, X_k^*).$$

假设引理 B.3、引理 B.4 和引理 B.5 成立证明引理 B.1： 我们将应用引理 B.3，令 δ 为一个足够小的常数（比如 $\delta = 0.01$）以采用联合界。引理 B.4 意味着第一个条件以 $D_1 = O(k^2)$ 成立，引理 B.5 意味着第二个条件以 $D_2 = 2^{O(z)}$ 成立。最后，引理 B.1 的第二个假设设定了 $D_3 = r$，从中我们得出结论。

我们现在证明引理 B.4 和引理 B.5。引理 B.3 在 B.2 小节中证明。

引理 B.4 的证明： 该证明依赖于高斯分布的 2-稳定性（2-stability）性质。即，如果我们令 $z \in \mathbb{R}^d$ 为任意向量，并采样标准高斯向量 $g \sim \mathcal{N}(0, I_d)$，则（标量）随机变量 $\langle z, g\rangle$ 的分布如同 $\|z\|_2 \cdot g'$，其中 $g' \sim \mathcal{N}(0, 1)$。利用高斯分布的 2-稳定性，对于每个 $i, j \in [k]$，我们有 $\|\Pi(c_i) - \Pi(c_j)\|_2^2$ 的分布如同 $(g')^2 \|c_i - c_j\|_2^2$，其中 g' 分布为（一维）高斯 $\mathcal{N}(0, 1)$。因此，通过联合界，存在一对 $i, j \in [k]$ 满足 $\|\Pi(c_i) - \Pi(c_j)\|_2^2 < \alpha^2 \cdot \|c_i - c_j\|_2^2$ 的概率至多为 k^2 乘以高斯随机变量位于 $[-\alpha, \alpha]$ 内的概率，且该概率显然小于 α。设置 $\alpha = \delta/k^2$ 即可得证。

引理 B.5 的证明： 与引理 B.4 的证明类似，我们有 $\|\Pi(x) - \Pi(c_i^*)\|_2^z$ 的分布如同 $|g'|^z \cdot \|x - c_i^*\|_2^z$，其中 g' 分布为（一维）高斯 $\mathcal{N}(0, 1)$。通过期望的线性性，

$$\mathbb{E}_{\Pi \sim J_d}\left[\sum_{i=1}^{k} \sum_{x \in X_i^*} \|\Pi(x) - \Pi(c_i^*)\|_2^z\right] = \sum_{i=1}^{k} \sum_{x \in X_i^*} \mathbb{E}_{g' \sim \mathcal{N}(0,1)}[|g'|^z] \cdot \|x - c_i^*\|_2^z.$$

为了总结，注意对于 $z \geq 1$，存在某个 $\alpha > 1$ 使得 αz 是偶整数且 $\alpha \leq 2$。因此，通过 Jensen 不等式和 $f(x) = x^{1/\alpha}$ 是凹函数的事实，我们可以写出

$$\mathbb{E}_{g' \sim \mathcal{N}(0,1)}[|g'|^z] \leq (\mathbb{E}[(g')^{\alpha z}])^{1/\alpha}.$$

现在注意，根据对称性，高斯分布的所有奇数矩均为零。因此，对于矩生成函数 $\mathbb{E}[e^{g'}]$ 成立

$$\mathbb{E}[e^{g'}] = \sum_{k=0}^{\infty} \frac{1}{(2k)!} \cdot \mathbb{E}[(g')^{2k}].$$

由于 $\mathbb{E}[e^{g'}] \le e^{1/2}$，由此可得

$$(\mathbb{E}[(g')^{\alpha z}])^{1/\alpha} \le ((\alpha z)!\mathbb{E}[e^{g'}])^{1/\alpha} \le ((\alpha z)!e^{1/2})^{1/\alpha} \le 2^{O(z)}.$$

应用马尔可夫不等式现在完成了证明。

B.2 引理 B.3 的证明 (Proof of Lemma B.3)

令 $\{\hat{c}_i\}_{i\in[k]}$ 为 $\Pi(X)$ 上 (k,z)-聚类问题的划分 $(Y_1,\dots,Y_k)$ 的最优中心集，其中 $\hat{c}_i \in \mathbb{R}^t$。具体来说，点 $\hat{c}_1,\dots,\hat{c}_k \in \mathbb{R}^t$ 是那些最小化下式的点

$$\mathrm{cost}(\Pi(Y_1),\dots,\Pi(Y_k)) \stackrel{\mathrm{def}}{=} \sum_{i=1}^{k}\sum_{x\in Y_i} \|\Pi(x)-\hat{c}_i\|_2^z.$$

为了量化中心 c^* 和中心 $\hat{c}$ 之间的代价差异，我们分析以下值。我们假设我们将 X 的每个点映射到 $\Pi c^*(X)$，然后我们计算该集合上划分 $Y_1,\dots,Y_k$ 的代价。形式上，我们令

$$\mathrm{val}(\Pi, c^*, Y_1,\dots,Y_k) = \sum_{i=1}^{k}\sum_{j=1}^{k} |Y_i \cap X_j^*| \cdot \|\Pi(c_j^*)-\hat{c}_i\|_2^z.$$

首先，我们证明以下简单的断言。

断言 B.6： 存在一组中心 $c_1',\dots,c_k'$（可能有重复），它们选自点集 $\{c_1^*,\dots,c_k^*\}$，使得

$$\sum_{i=1}^{k}\sum_{j=1}^{k} |Y_i \cap X_j^*| \cdot \|\Pi(c_j^*)-\Pi(c_i')\|_2^z \le 2^z \cdot \mathrm{val}(\Pi, c^*, Y_1,\dots,Y_k).$$

证明： 我们将使用概率法证明该断言。对于每个 $i\in[k]$，考虑中心 $\{c_1^*,\dots,c_k^*\}$ 上的分布，该分布采样中心 c_i' 的概率为

$$\Pr_{c_i'}[c_i' = c_j^*] = \frac{|Y_i \cap X_j^*|}{\sum_{\ell=1}^{k} |Y_i \cap X_\ell^*|}.$$

然后，我们上界使用中心 c_i' 的期望代价。使用引理 A.3，

$$\mathbb{E}\left[\sum_{i=1}^{k}\sum_{j=1}^{k} |Y_i \cap X_j^*| \cdot \|\Pi(c_j^*)-\Pi(c_i')\|_2^z\right] \le \sum_{i=1}^{k}\sum_{j=1}^{k} |Y_i \cap X_j^*| \cdot \mathbb{E}[\|\Pi(c_j^*)-\hat{c}_i\|_2^z + \|\Pi(c_i')-\hat{c}_i\|_2^z] \cdot 2^{z-1}$$

（此处省略了部分中间推导步骤，直接给出结论）

$$\le 2^z \cdot \mathrm{val}(\Pi, c^*, Y_1,\dots,Y_k).$$

我们现在根据 $\mathrm{cost}_z(X_1^*,\dots,X_k^*)$ 上界 $\mathrm{cost}_z(Y_1,\dots,Y_k)$。我们通过使用根据断言 B.6 选择的中心来做到这一点。这将允许我们根据 $\mathrm{cost}_z(X_1^*,\dots,X_k^*)$ 以及仅涉及 $\{c_1^*,\dots,c_k^*\}$ 之间成对距离的聚类代价来上界 $(Y_1,\dots,Y_k)$ 的聚类代价。然后我们将这些距离关联到应用映射 Π 后的距离。具体来说，首先注意如果我们考虑从断言 B.6 选择的中心集 $c_1',\dots,c_k'$，

$$\mathrm{cost}_z(Y_1,\dots,Y_k) \le \sum_{i=1}^{k}\sum_{x\in Y_i} \|x-c_i'\|_2^z \le 2^{z-1}\cdot \mathrm{cost}_z(X_1^*,\dots,X_k^*) + 2^{z-1}\sum_{i=1}^{k}\sum_{j=1}^{k} |Y_i \cap X_j^*| \cdot \|c_j^*-c_i'\|_2^z, \quad (5)$$

其中第三个不等式再次使用了引理 A.3。注意 (5) 的最右侧求和涉及仅在 $c_1^*,\dots,c_k^*$ 之间的距离，因此根据映射 Π 的第一个假设和断言 B.6，我们可以上界

$$\sum_{i=1}^{k}\sum_{j=1}^{k} |Y_i \cap X_j^*| \cdot \|c_j^*-c_i'\|_2^z \le D_1^z \sum_{i=1}^{k}\sum_{j=1}^{k} |Y_i \cap X_j^*| \cdot \|\Pi(c_j^*)-\Pi(c_i')\|_2^z \le D_1^z \cdot 2^z \cdot \mathrm{val}(\Pi, c^*, Y_1,\dots,Y_k). \quad (6)$$

结合 (5) 和 (6)，我们可以上界

$$\mathrm{cost}_z(Y_1,\ldots,Y_k) \le 2^{z-1}\cdot \mathrm{cost}_z(X_1^*,\ldots,X_k^*) + D_1^z \cdot 2^{2z-1}\cdot \mathrm{val}(\Pi, c^*, Y_1,\ldots,Y_k).$$

我们继续通过应用三角不等式上界 (6) 中最右侧的表达式

$$\begin{aligned}
&\sum_{i=1}^k\sum_{j=1}^k |Y_i\cap X_j^*|\cdot \|\Pi(c_j^*)-\hat{c}_i\|_2^z \\
&\le 2^{z-1}\sum_{i=1}^k\sum_{j=1}^k\sum_{x\in Y_i\cap X_j^*}\|\Pi(x)-\Pi(c_j^*)\|_2^z + 2^{z-1}\sum_{i=1}^k\sum_{j=1}^k\sum_{x\in Y_i\cap X_j^*}\|\Pi(x)-\hat{c}_i\|_2^z \\
&\le 2^{z-1}D_2\cdot \mathrm{cost}_z(X_1^*,\ldots,X_k^*) + 2^{z-1}\cdot \mathrm{cost}_z(\Pi(Y_1),\ldots,\Pi(Y_k)). \quad (7)
\end{aligned}$$

根据引理的第三个假设，我们注意到

$$\begin{aligned}
&\mathrm{cost}_z(\Pi(Y_1),\ldots,\Pi(Y_k)) \le D_3\cdot \min_{c_1,\ldots,c_k\in\mathbb{R}^t}\sum_{x\in X}\min_{j\in[k]}\|\Pi(x)-c_j\|_2^z \\
&\le D_3\cdot\sum_{j=1}^k\sum_{x\in X_j^*}\|\Pi(x)-\Pi(c_j^*)\|_2^z \le D_3D_2\cdot \mathrm{cost}_z(X_1^*,\ldots,X_k^*). \quad (8)
\end{aligned}$$

通过将 (7) 和 (8) 代入 (6) 进行总结，我们可以上界

$$\mathrm{cost}_z(Y_1,\ldots,Y_k) \le \left(2^{3z-1}D_1^z(1+D_3)+2^{2z-1}\right)D_2\cdot \mathrm{cost}_z(X_1^*,\ldots,X_k^*).$$

C. 一维中的高效初始化 (Efficient Seeding in One Dimension)

在本节中，我们证明定理 2.1，该定理给出了我们算法 PRONE 运行时间的上界。正如在定理 B.2 中一样，我们考虑 PRONE 的推广版本，其中我们在第 2 步运行针对一般 $z \ge 1$ 的 k-means++ 初始化（定义 A.2）。我们证明以下定理 2.1 的推广版本。

定理 C.1 (针对一般 $z \ge 1$ 的定理 2.1)： 令 $X = \{x_1,\ldots,x_n\}$ 为包含 d 维点的数据集。假设 $d \le \mathrm{nnz}(X)$，这可以通过移除冗余维度（即每个 x_i 的第 j 个坐标均为零的维度 $j \in [d]$）来确保。对于任意 $z \ge 1$，算法 PRONE（针对一般 z 如定理 B.2 所述）在 X 上的期望运行时间为 $O(\mathrm{nnz}(X) + 2^{z/2}n\log n)$。对于任意 $\delta \in (0,1/2)$，算法以至少 $1-\delta$ 的概率在 $O(\mathrm{nnz}(X) + 2^{z/2}n\log(n/\delta))$ 时间内运行。此外，算法总是运行在 $O(\mathrm{nnz}(X) + n\log n + nk)$ 时间内。

为了证明定理 2.1，我们展示了 k-means++ 初始化过程的一种高效实现（算法 1），它对于一维点在期望时间 $O(2^{z/2}n\log n)$ 内运行（引理 C.3）。初始化过程的朴素实现将在一维中花费 $\Theta(nk)$ 时间，因为我们需要 $\Theta(n)$ 时间来更新 p_i 并从 D^z 分布中采样以添加每个 k 个中心。为了获得改进且可证明的运行时间，我们使用基本的二叉树数据结构来更有效地从 D^z 分布中采样，并使用势能参数（potential argument）来界定 p_i 的更新次数。

我们在算法 1 中使用的数据结构可以实现为基本的二叉树，详见附录 D。该数据结构跟踪对应于 $p_1^z,\ldots,p_n^z$ 的 n 个非负数 $s_1,\ldots,s_n$，并支持以下操作：

1. **Initialize(a)：** 给定数组 $a = (a_1,\ldots,a_n) \in \mathbb{R}^n_{\ge 0}$，操作 Initialize($a$) 创建一个数据结构，跟踪初始化为 $(s_1,\ldots,s_n) = (a_1,\ldots,a_n)$ 的数字 $s_1,\ldots,s_n$。此操作运行时间为 $O(n)$。
2. **Sum(S)：** 操作 Sum(S) 返回总和 $\sum_{i=1}^n s_i$。此操作运行时间为 $O(1)$，因为该值将在数据结构更新时被维护。
3. **Find(S, r)：** 给定数字 $r \in [0, \sum_{i=1}^n s_i)$，操作 Find($S, r$) 返回唯一的索引 $\ell \in \{1,\ldots,n\}$ 使得 $\sum_{i=1}^{\ell-1} s_i \le r < \sum_{i=1}^{\ell} s_i$。此操作运行时间为 $O(\log n)$。
4. **Update(S, a, i_1, i_2)：** 给定数组 $a = (a_1,\ldots,a_n) \in \mathbb{R}^n_{\ge 0}$ 和满足 $1 \le i_1 \le i_2 \le n$ 的索引 i_1, i_2，操作 Update(S, a, i_1, i_2) 对每个 $\ell \in \{i_1,\ldots,i_2\}$ 执行更新 $s_\ell \leftarrow a_\ell$。此操作运行时间为 $O((i_2 - i_1 + 1)\log n)$。

算法 1：一维中的高效 k-means++ 初始化

Algorithm 1: Efficient k-means++ seeding in one dimension

Input: Points $x_1, \ldots, x_n \in \mathbb{R}$; $k \in \mathbb{Z}$ satisfying $1 \le k \le n$; real number $z \ge 1$.
Output: Centers $x_{\ell_1}, \ldots, x_{\ell_k} \in \mathbb{R}$; assignment $\sigma : [n] \to [k]$.

1 Sort and re-order the points so that $x_1 \le \cdots \le x_n$;
2 Choose ℓ_1 uniformly at random from $\{1, \ldots, n\}$;
3 Initialize $a = (a_1, \ldots, a_n)$ by setting $a_i \leftarrow |x_i - x_{\ell_1}|^z$ for every $i = 1, \ldots, n$;
4 $S \leftarrow \texttt{Initialize}(a)$;
5 **for** $t = 2, \ldots, k$ **do**
6 Choose r uniformly at random from $[0, \texttt{Sum}(S))$;
7 $\ell_t \leftarrow \texttt{Find}(S, r)$; $a_{\ell_t} \leftarrow 0$; $i \leftarrow \ell_t - 1$; $j \leftarrow \ell_t + 1$;
8 **while** $i \ge 0$ **and** $|x_i - x_{\ell_t}|^z < a_i$ **do**
9 $a_i \leftarrow |x_i - x_{\ell_t}|^z$;
10 $i \leftarrow i - 1$;
11 **end**
12 **while** $j \le n$ **and** $|x_j - x_{\ell_t}|^z < a_j$ **do**
13 $a_j \leftarrow |x_j - x_{\ell_t}|^z$;
14 $j \leftarrow j + 1$;
15 **end**
16 $\texttt{Update}\ (S, a, i+1, j-1)$;
17 **end**
18 Sort and re-order $\ell_1, \ldots, \ell_k$ so that $\ell_1 \le \cdots \le \ell_k$;
19 $i \leftarrow 1$; $j \leftarrow 1$;
20 **while** $i \le n$ **do** /* Assign x_i to the closest center $x_{\ell_{\sigma(i)}}$ among $x_{\ell_1}, \ldots, x_{\ell_k}$. */
21 **if** $j < k$ **and** $|x_i - x_{\ell_j}| \ge |x_i - x_{\ell_{j+1}}|$ **then**
22 $j \leftarrow j + 1$;
23 **else**
24 $\sigma(i) \leftarrow j$;
25 $i \leftarrow i + 1$;
26 **end**
27 **end**
28 **return** $x_{\ell_1}, \ldots, x_{\ell_k}$ and σ (converted to the old ordering of $x_1, \ldots, x_n$ before Line 1);

以下断言表明算法 1 正确地在一维中实现了 k-means++ 初始化过程。

断言 C.2： 考虑 for 循环每次迭代开始时（即第 6 行之前）$t, a_1, \ldots, a_n$ 的值和数据结构 S。令 $s_1, \ldots, s_n$ 为数据结构 S 跟踪的数字。对于每个 $i = 1, \ldots, n$，定义 $p_i := \min_{t'=1,\ldots,t-1} |x_i - x_{\ell_{t'}}|$。则对于每个 $i = 1, \ldots, n$，有 $s_i = a_i = p_i^z$。因此，在第 7 行以迄今为止的执行历史为条件的 ℓ_t 的分布满足 $\Pr[\ell_t = i] = p_i^z / \sum_{i'=1}^n p_{i'}^z$ 对每个 $i = 1, \ldots, n$ 成立。

该断言通过基于上述数据结构 S 及其操作描述对 for 循环的迭代进行归纳直接得出。以下引理界定了算法 1 的运行时间：

引理 C.3 (针对一般 $z \ge 1$ 的引理 2.4)： 算法 1 的期望运行时间为 $O(2^{z/2} n \log n)$。对于任意 $\delta \in (0, 1/2)$，算法 1 以至少 $1 - \delta$ 的概率在 $O(2^{z/2} n \log(n/\delta))$ 时间内运行。此外，算法 1 总是运行在 $O(n \log n + nk)$ 时间内。

在证明引理 C.3 之前，我们先用它来证明定理 C.1。

定理 C.1 的证明： 引理 C.3 界定了第 2 节中定义的算法 PRONE 第 2 步的运行时间。现在我们展示第 1 步（随机一维投影）可以在 $O(\mathrm{nnz}(X) + n)$ 时间内执行。确实，x'_i 可以计算为 $x'_i = \sum_j x_{ij} v_j$，其中求和是对 x_i 的所有非零坐标 x_{ij} 进行的，且每个 v_j 独立地从一维标准高斯中抽取（v_j 的值应对所有 i 共享）。以这种方式计算 $x'_1, \ldots, x'_n$ 所需的时间为 $O(\mathrm{nnz}(X) + n)$。在 PRONE 的第 3 步中，我们计算每个簇的质心。这可以通过对每个簇中的点求和并将每个总和除以该簇中的点数来在 $O(\mathrm{nnz}(X) + n)$ 时间内完成。

证明引理 C.3 的关键步骤是界定第 9 行和第 13 行对 a 的更新次数。由于算法开始时对 n 个输入点进行排序，耗时 $O(n \log n)$，我们可以假设点已排序使得 $x_1 \le \cdots \le x_n$。对于 $i = 1, \ldots, n$ 和 $t = 2, \ldots, k$，我们定义 $\xi(i, t) = 1$ 如果 a_i 在迭代 t 的第 9 行被更新，否则定义 $\xi(i, t) = 0$。这里，我们用迭代变量 t 的值表示从第 5 行开始的 for 循环的每次迭代。我们定义 $u_i := \sum_{t=2}^k \xi(i, t)$ 为 a_i 在第 9 行被更新的次数。以下引理给出了 u_i 在期望和高概率下的上界：

引理 C.4： 对于每个 $i = 1, \ldots, n$，成立

$$\mathbb{E}[u_i] \le O(2^{z/2} \log n).$$

此外，对于某个绝对常数 $B > 0$ 和任意 $\delta \in (0, 1/2)$，成立

$$\Pr[u_i \le B2^{z/2}\log(n/\delta)] \ge 1-\delta.$$

在证明引理 C.4 之前，我们先用它来证明引理 C.3。

引理 C.3 的证明： 回顾对于 $i=1,\dots,n$ 和 $t=2,\dots,k$，我们定义 $\xi(i,t)=1$ 如果 a_i 在迭代 t 的第 9 行被更新，否则 $\xi(i,t)=0$。类似地，对于 $j=1,\dots,n$ 和 $t=2,\dots,k$，我们定义 $\xi'(j,t)=1$ 如果 a_j 在迭代 t 的第 13 行被更新，否则 $\xi'(j,t)=0$。

第 1-4 行的计算耗时 $O(n\log n)$。第 18-28 行的计算耗时 $O(k\log k+n)=O(n\log n)$。对于 $t=2,\dots,k$，for 循环的迭代 t 耗时 $O(\log n+\sum_{i=1}^n \xi(i,t)+\sum_{i=1}^n \xi'(i,t))$。将它们求和，算法 1 的总运行时间为

$$O\left(n\log n+\sum_{i=1}^n\sum_{t=2}^k \xi(i,t)+\sum_{i=1}^n\sum_{t=2}^k \xi'(i,t)\right). \quad (9)$$

根据引理 C.4，

$$\mathbb{E}\left[\sum_{i=1}^n\sum_{t=2}^k \xi(i,t)\right]=\mathbb{E}\left[\sum_{i=1}^n u_i\right]=O(2^{z/2}n\log n). \quad (10)$$

此外，对于任意 $\delta'\in(0,1/2)$，在引理 C.4 中设置 $\delta=\delta'/n$，通过联合界我们有

$$\Pr\left[\sum_{i=1}^n\sum_{t=2}^k \xi(i,t)\le 2B2^{z/2}n\log(n/\delta')\right]\ge \Pr\left[\sum_{i=1}^n\sum_{t=2}^k \xi(i,t)\le B2^{z/2}n\log(n/\delta)\right]\ge 1-n\delta=1-\delta'. \quad (11)$$

类似于 (10) 和 (11)，我们有

$$\mathbb{E}\left[\sum_{i=1}^n\sum_{t=2}^k \xi'(i,t)\right]=O(2^{z/2}n\log n), \quad (12)$$

以及

$$\Pr\left[\sum_{i=1}^n\sum_{t=2}^k \xi'(i,t)\le 2B2^{z/2}n\log(n/\delta')\right]\ge 1-\delta'. \quad (13)$$

将 (10) 和 (11) 代入 (9) 证明了算法 1 的期望运行时间为 $O(2^{z/2}n\log n)$。选择 $\delta'=\delta/2$ 作为引理 C.3 中的 δ 并将 (11) 和 (13) 代入 (9)，我们可以使用联合界得出结论：算法 1 以至少 $1-\delta$ 的概率在 $O(2^{z/2}n\log(n/\delta))$ 时间内运行。最后，将 $\xi(i,t)\le 1$ 和 $\xi'(i,t)\le 1$ 代入 (9)，我们得到算法 1 总是运行在 $O(n\log n+nk)$ 时间内。

为了证明引理 C.4，对于 $i=0,\dots,n$ 和 $u=0,\dots,u_i$，我们定义 $t(i,u)$ 为最小的 $t\in\{1,\dots,k\}$ 使得 $\sum_{t'=2}^t \xi(i,t')=u$。即，$a_i$ 在迭代 $t(i,u)$ 中第 u 次在第 9 行被更新。我们的定义意味着 $t(i,0)=1$ 且对于 $u=1,\dots,u_i$ 有 $t(i,u)\in\{2,\dots,k\}$。我们定义一个非负势能函数 $\eta(i,u)$ 如下，并证明它随着 u 的增加呈指数级下降（引理 C.5）。

势能函数 $\eta(i,u)$： 对于 $t=2,\dots,k$，我们考虑迭代 t 中第 7 行之后 ℓ_t 的值。对于 $u=1,\dots,u_i$，我们定义 $\eta(i,u)$ 为 $\ell_{t(i,u)}-i$，根据 $t(i,u)$ 的定义，这保证是一个正整数。确实，在包含第 9 行的 while 循环中，i 从 ℓ_t-1 开始并保持递减，所以每当执行第 9 行时，i 小于 ℓ_t。特别地，a_i 在 for 循环的迭代 $t(i,u)$ 中第 9 行被更新，所以我们有 $i<\ell_{t(i,u)}$。我们定义 $\eta(i,0)=n$，对于 $u=u_i+1,u_i+2,\dots$，我们定义 $\eta(i,u)=0$。见图 4 说明 $\eta(i,u)$ 的定义。

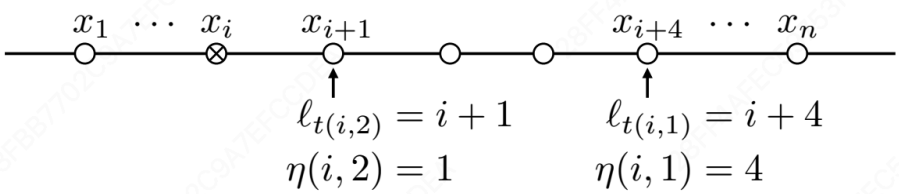


图 4：说明势能函数 η 定义的示例。这里，a_i 在迭代 $t(i,1)$ 中第一次在第 9 行被更新，此时 $\ell_{t(i,1)}=i+4$。然后 a_i 在迭代 $t(i,2)$ 中第二次在第 9 行被更新，此时 $\ell_{t(i,2)}=i+1$。每当 a_i 在第 9 行被更新时，我们总是有 $\ell_t>i$，η 是 ℓ_t 和 i 之间的差值。因此在这个例子中 $\eta(i,1)=4$ 且 $\eta(i,2)=1$。如果在迭代 $t(i,2)$ 之后 a_i 再也没有在第 9 行被更新，我们定义 $\eta(i,u)=0$ 对于 $u=3,4,\dots$。

引理 C.5 (势能函数下降)： 对于任意 $i\in\{1,\dots,n\}$ 和 $u\in\mathbb{Z}_{\ge 0}$，

$$\mathbb{E}[\eta(i,u+1)|\eta(i,0),\ldots,\eta(i,u)] \le \max\left\{0, \frac{2^{z/2}}{2^{z/2}+1}\eta(i,u) - \frac{1}{2}\right\}.$$

在证明引理 C.5 之前，我们先用它来证明引理 C.4。直观地说，引理 C.5 表明 $\eta(i,u)$ 作为 u 的函数呈指数级下降（在期望上）。由于 $\eta(i,u)$ 始终是非负整数，我们应该预期一旦 u 超过一个小阈值，$\eta(i,u)$ 就会变为零。此外，我们的定义确保 $\eta(i,u_i) > 0$，所以 u_i 必须小于该阈值。这允许我们展示 u_i 的上界并证明引理 C.4。

引理 C.4 的证明： 我们的 η 定义确保 $\eta(i,u_i) \ge 1$。根据引理 C.5 和引理 C.7，

$$\mathbb{E}[u_i+1] \le \frac{\ln n}{\ln \frac{2^{z/2+1}}{2^{z/2}}} + \frac{1}{1-\frac{2^{z/2}}{2^{z/2+1}}} = O(2^{z/2}\ln n) + 2^{z/2+1}.$$

这意味着 $\mathbb{E}[u_i] = O(2^{z/2}\log n)$。此外，根据引理 C.5，

$$\mathbb{E}[\eta(i,u)] \le \eta(i,0)\left(\frac{2^{z/2}}{2^{z/2}+1}\right)^u = n\left(\frac{2^{z/2}}{2^{z/2}+1}\right)^u,$$

因此，根据马尔可夫不等式，

$$\Pr[u_i \ge u] = \Pr[\eta(i,u) \ge 1] \le n\left(\frac{2^{z/2}}{2^{z/2}+1}\right)^u.$$

对于任意 $\delta \in (0,1/2)$，在上述不等式中选择 $u = \ln(n/\delta)/\ln(\frac{2^{z/2+1}}{2^{z/2}}) = O(2^{z/2}\log(n/\delta))$ 给出 $\Pr[u_i \ge u] \le \delta$。

我们需要以下辅助引理来证明引理 C.5。

引理 C.6： 在算法 1 的 for 循环每次迭代开始时（即第 6 行执行之前），以下情况成立。选择任意 $i = 1,\ldots,n$ 并定义

$$L := \{i\} \cup \{\ell \in \mathbb{Z} : i < \ell \le n, |x_i - x_\ell|^z < a_i\}. \quad (14)$$

则对于满足 $\ell < \ell'$ 的 $\ell, \ell' \in L$，成立 $a_{\ell'} \le 2^z a_\ell$。

证明： 对于 $t = 2,\ldots,k$，在迭代 t 开始时，值 $\ell_1,\ldots,\ell_{t-1}$ 已经被确定。对于每个 $x \in \mathbb{R}$，令 $\rho(x)$ 表示 $x_{\ell_1},\ldots,x_{\ell_{t-1}}$ 中最接近 x 的值。根据断言 C.2，对于每个 $i \in [n]$，$a_i = |x_i - \rho(x_i)|^z$。现在对于固定的 $i \in [n]$，如 (14) 定义 L 并考虑满足 $\ell < \ell'$ 的 $\ell, \ell' \in L$。很容易看出 $x_i \le \rho(x_\ell) \le x_{\ell'}$ 不可能成立，否则 $a_i \le |x_i - \rho(x_\ell)|^z \le |x_i - x_{\ell'}|^z < a_i$，矛盾。同理，$x_i \le \rho(x_{\ell'}) \le x_{\ell'}$ 也不可能成立。因此 $x_i, x_\ell, x_{\ell'}, \rho(x_\ell), \rho(x_{\ell'})$ 只有三种可能的顺序：

1. $\rho(x_\ell) = \rho(x_{\ell'}) < x_i \le x_\ell \le x_{\ell'}$；
2. $\rho(x_\ell) < x_i \le x_\ell \le x_{\ell'} < \rho(x_{\ell'})$；
3. $x_i \le x_\ell \le x_{\ell'} < \rho(x_\ell) = \rho(x_{\ell'})$。

在情形 3 中，显然 $a_{\ell'} = |x_{\ell'} - \rho(x_{\ell'})|^z \le |x_\ell - \rho(x_\ell)|^z = a_\ell$。在前两种情形中，对于任意 $t' = 0,\ldots,t-1$，

$$|x_i - x_{\ell_{t'}}| \ge |x_\ell - x_{\ell_{t'}}| - |x_\ell - x_i| \ge |x_\ell - \rho(x_\ell)| - |x_\ell - x_i| = |x_i - \rho(x_\ell)|.$$

这意味着 $\rho(x_\ell)$ 是 $x_{\ell_1},\ldots,x_{\ell_{t-1}}$ 中最接近 x_i 的点。因此，$a_i = |x_i - \rho(x_\ell)|^z$。
Jensen 不等式确保对于任意 $g, h \ge 0$，((g + h)/2)^z \le (g^z + h^z)/2，这意味着(g + h)^z \le 2^{z-1} g^z + 2^{z-1} h^z。因此，$a_{\ell'} \le |x_{\ell'} - \rho(x_\ell)|^z \le 2^{z-1}|x_{\ell'} - x_i|^z + 2^{z-1}|x_i - \rho(x_\ell)|^z < 2^z a_i$,而$a_\ell = |x_\ell - \rho(x_\ell)|^z \ge |x_i - \rho(x_\ell)|^z = a_i$.因此，在所有三种情形下我们都有 a_{\ell'} \le 2^z a_\ell$。

引理 C.5 的证明： 在整个证明过程中，我们固定 $i \in \{1,\ldots,n\}$ 和 $u \in \mathbb{Z}_{\ge 0}$ 为确定性数字。算法 1 是随机算法，当我们运行它时，恰好发生以下四个事件之一，我们据此定义随机变量 t^*：

1. **事件 E1：** $u_i < u$。即 a_i 在第 9 行被更新少于 u 次。在这种情况下，根据 η 的定义我们有 $\eta(i,u+1) = 0$，我们定义 $t^* = +\infty$。
2. **事件 E2：** $u_i = u$ 且 i 从未在第 7 行被选为 ℓ_t。在这种情况下我们也由 $\eta(i,u+1) = 0$，我们也定义 $t^* = +\infty$。
3. **事件 E3：** $u_i = u$ 且存在 $t \in \{2,3,\ldots,k\}$ 使得 i 在迭代 t 的第 7 行被选为 ℓ_t。此 t 必须满足 $t > t(i,u)$，因为第 9 行对 a_i 的所有更新必须发生在 x_i 被选为中心之前。在这种情况下我们定义 $t^* = t$。同样，在这种情况下我们有 $\eta(i,u+1) = 0$。
4. **事件 E4：** $u_i > u$。在这种情况下我们定义 $t^* := t(i,u+1) > t(i,u)$。

定义 $E^* := E3 \cup E4$。由于在 E1 和 E2 下 $\eta(i,u+1) = 0$，证明以下内容就足够了：

$$\mathbb{E}[\eta(i,u+1)|\eta(i,0),\ldots,\eta(i,u),E^*] \le \frac{2^{z/2}}{2^{z/2}+1}\eta(i,u) - \frac{1}{2}. \quad (15)$$

根据我们的定义，随机变量 t^* 取值于 $\{2,3,\ldots,k\}\cup\{+\infty\}$。此外，$t^*=+\infty$ 当且仅当 E^* 不发生。因此，为了证明 (15)，只需证明对于每个 $t_0=2,3,\ldots,k$：

$$\mathbb{E}[\eta(i,u+1)|\eta(i,0),\ldots,\eta(i,u),t^*=t_0]\le\frac{2^{z/2}}{2^{z/2}+1}\eta(i,u)-\frac{1}{2}. \quad (16)$$

考虑固定的 $t_0\in\{2,3,\ldots,k\}$。要发生 $t^*=t_0$，在算法 1 执行迭代 t_0 之前必须满足以下条件：a_i 已经在第 9 行被更新了恰好 u 次，且 i 尚未在第 7 行被选为 ℓ_t。因此，证明的其余部分假设算法 1 在迭代 t_0 之前的执行历史 H 满足此属性。现在我们知道值 $\eta(i,0),\ldots,\eta(i,u)$ 由执行历史 H 确定。第 6-7 行保证 ℓ_{t_0} 的分布满足

$$\Pr[\ell_{t_0}=\ell|H]=\frac{a_\ell}{\sum_{j=1}^n a_j}\quad \text{对于每个 } \ell=1,\ldots,n,$$

其中我们使用迭代 t_0 执行之前的 $a_1,\ldots,a_n$ 值。此外，以 H 为条件，我们有 $t^*=t_0$ 当且仅当 $\ell_{t_0}\in L$，其中

$$L=\{i\}\cup\{\ell\in\mathbb{Z}:i<\ell\le n,|x_\ell-x_i|^z<a_i\}.$$

因此，如果我们进一步以 $t^*=t_0$ 为条件，我们有 $\ell_{t_0}\in L$ 且

$$\Pr[\ell_{t_0}=\ell|H,t^*=t_0]=\frac{a_\ell}{\sum_{j\in L}a_j}\quad \text{对于每个 } \ell\in L. \quad (17)$$

当 $t^*=t_0$ 时，我们有 $\eta(i,u+1)=\ell_{t_0}-i$。因此，为了证明 (16)，只需证明

$$\mathbb{E}[\ell_{t_0}-i|H,t^*=t_0]\le\frac{2^{z/2}}{2^{z/2}+1}\eta(i,u)-\frac{1}{2}. \quad (18)$$

显然我们可以将 L 写为 $L=\{i,i+1,\ldots,\ell^*\}$ 对于某个整数 $\ell^*\ge i$。如果 $u>0$，断言 C.2 意味着 $a_i\le|x_i-x_{\ell_{t(i,u)}}|^z$，因此 $\ell^*<\ell_{t(i,u)}$ 且 $\ell^*-i\le\ell_{t(i,u)}-i=\eta(i,u)$。如果 $u=0$，我们有 $\eta(i,u)=n$，所以也成立 $\ell^*-i\le\eta(i,u)$。根据 (17) 和引理 C.6，我们可以在引理 C.8 中设置 $\gamma=2^z$ 得到

$$\mathbb{E}[\ell_{t_0}-i|H,t^*=t_0]\le\frac{2^{z/2}}{2^{z/2}+1}(\ell^*-i)-\frac{1}{2}\le\frac{2^{z/2}}{2^{z/2}+1}\eta(i,u)-\frac{1}{2}.$$

这证明了 (18)，从而证明了引理。

C.1 辅助引理 (Helper Lemmas)

引理 C.7： 令 $M\ge1$ 和 $\lambda\in(0,1)$ 为参数。令 $\alpha_0,\alpha_1,\cdots\in[0,+\infty)$ 为随机变量，满足 $\alpha_0=M$ 且对于每个 $i=0,1,\ldots$ 成立 $\mathbb{E}[\alpha_{i+1}|\alpha_1,\ldots,\alpha_i]\le\lambda\alpha_i$。令 $t\ge0$ 为满足 $\alpha_t<1$ 的最小整数。则

$$\mathbb{E}[t]\le\frac{\ln M}{\ln(1/\lambda)}+\frac{1}{1-\lambda}.$$

证明： 对于每个 $j=0,1,\ldots$，我们定义随机变量 $t_j:=\min\{t,j\}$。根据单调收敛定理，只需证明

$$\mathbb{E}[t_j]\le\frac{\ln M}{\ln(1/\lambda)}+\frac{1}{1-\lambda}\quad \text{对于每个 } j=0,1,\ldots. \quad (19)$$

我们通过对 j 归纳来证明 (19)。当 $j=0$ 时，我们有 $t_j=0$，上述不等式平凡成立。我们假设 (19) 对任意 $j\in\mathbb{Z}_{\ge0}$ 成立，并证明它对 $j+1$ 也成立。我们有

$$\mathbb{E}[t_{j+1}]=1+\mathbb{E}[t_{j+1}-1]=1+\mathbb{E}[\mathbb{E}[(t_{j+1}-1)|\alpha_1]]. \quad (20)$$

根据 t_{j+1} 的定义，我们有 $t_{j+1}-1=\min\{t-1,j\}$。对序列 $\alpha_1,\alpha_2,\ldots$ 应用我们的归纳假设，我们有

$$\mathbb{E}[(t_{j+1}-1)|\alpha_1]=\mathbb{E}[\min\{t-1,j\}|\alpha_1]\le f(\alpha_1), \quad (21)$$

其中

$$f(\alpha)=\begin{cases}\frac{\alpha}{1-\lambda}, & \text{如果 } \alpha\in[0,1);\\ \frac{\ln\alpha}{\ln(1/\lambda)}+\frac{1}{1-\lambda}, & \text{如果 } \alpha\geq 1.\end{cases}$$

很容易验证 f 是 $\alpha\in[0,+\infty)$ 的增凹函数，且对于每个 $\alpha\geq 1$ 成立 $1+f(\lambda\alpha)\leq f(\alpha)$。将 (21) 代入 (20)，我们有

$$\mathbb{E}[t_{j+1}]\leq 1+\mathbb{E}[f(\alpha_1)]\leq 1+f(\mathbb{E}[\alpha_1])\leq 1+f(\lambda M)\leq f(M)=\frac{\ln M}{\ln(1/\lambda)}+\frac{1}{1-\lambda}.$$

引理 C.8： 令 $\gamma\geq 1$ 为实数。令 $\beta_0,\ldots,\beta_{m-1}$ 为非负实数，使得对于每个满足 $i\leq j$ 的 $i,j\in\{0,\ldots,m-1\}$，成立 $\beta_j\leq\gamma\beta_i$。则，

$$\sum_{i=0}^{m-1}i\beta_i\leq\left(\frac{\sqrt{\gamma}\cdot m}{\sqrt{\gamma}+1}-\frac{1}{2}\right)\sum_{i=0}^{m-1}\beta_i.$$

证明： 如果 $\beta_0=0$，引理平凡成立，因为在这种情况下对于每个 $j=0,\ldots,m-1$ 有 $\beta_j\leq\gamma\beta_0=0$。因此我们不失一般性地假设 $\beta_0>0$。定义 τ 为满足下式的唯一实数

$$\tau\sum_{i=0}^{m-1}\beta_i-\sum_{i=0}^{m-1}i\beta_i=0.$$

显然 $\tau\in[0,m-1]$。我们的目标是证明

$$\tau\leq\frac{\sqrt{\gamma}\cdot m}{\sqrt{\gamma}+1}-\frac{1}{2}. \quad (22)$$

定义 $\beta^*:=\min_{0\leq i\leq\tau}\beta_i$。对于每个 $i=0,\ldots,m-1$，如果 $i\leq\tau$，我们有 $\beta_i\geq\beta^*$，如果 $i>\tau$，我们有 $\beta_i\leq\gamma\beta^*$。因此，定义 $s:=\lfloor\tau\rfloor$，我们有

$$\begin{aligned}0&=\tau\sum_{i=0}^{m-1}\beta_i-\sum_{i=0}^{m-1}i\beta_i=\sum_{i=0}^{m-1}(\tau-i)\beta_i\\ &\geq\sum_{i\leq\tau}(\tau-i)\beta^*+\sum_{i>\tau}(\tau-i)\gamma\beta^* \quad (23)\\ &=\frac{(s+1)(2\tau-s)}{2}\cdot\beta^*+\frac{(m-s-1)(2\tau-m-s)}{2}\cdot\gamma\beta^*. \quad (24)\end{aligned}$$

现在，我们证明 $\beta^*>0$。为了反证，假设 $\beta^*=0$。我们已经假设 $\beta_0>0$，所以 $\beta^*\neq\beta_0$。根据 β^* 的定义，这意味着 $\tau>0$ 且不等式 (23) 是严格的，导致错误的结论

$$0>\sum_{i\leq\tau}(\tau-i)\beta^*+\sum_{i>\tau}(\tau-i)\gamma\beta^*=0.$$

因此，必须成立 $\beta^*>0$。现在我们知道 (24) 意味着

$$(s+1)(2\tau-s)+(m-s-1)(2\tau-m-s)\gamma\leq 0.$$

将 s 视为实值变量，左侧在 $s=\tau-1/2$ 时最小化，给我们

$$(\tau+1/2)^2-(m-\tau-1/2)^2\gamma\leq 0.$$

上述不等式意味着

$$(\tau+1/2)^2\leq(m-\tau-1/2)^2\gamma.$$

两边开平方并解出 τ 给出 (22)。

D. 用于初始化中快速采样的数据结构 (Data Structure for Fast Sampling in Seeding)

在附录 C 中，我们的算法 1 使用了一个二叉树数据结构 S，它跟踪 n 个非负数 $s_1, \ldots, s_n$ 并支持多种操作。在这里，我们描述该数据结构的实现。我们假设 $n = 2^q$ 对于某个非负整数 q。这不失一般性，因为我们可以选择满足 $n \le n' < 2n$ 且 $n' = 2^q$（对于某个 $q \in \mathbb{Z}_{\ge 0}$）的数字 n'，并考虑 $s_1, \ldots, s_n, s_{n+1}, \ldots, s_{n'}$，其中 $s_{n+1} = \cdots = s_{n'} = 0$。在此假设下，数据结构 S 是一个具有 $q+1$ 层（索引为 $0, \ldots, q$）的完全二叉树。在每一层 $\zeta = 0, \ldots, q$ 中，有 2^ζ 个节点，每个节点对应于 $\{1, \ldots, n\}$ 中的一组索引。根节点记为 $v_1^{(0)}$，是第 0 层中的唯一节点，它对应于整个集合 $V_1^{(0)} := \{1, \ldots, n\}$。

对于 $\zeta = 0, \ldots, q-1$，第 ζ 层的每个节点 $v_j^{(\zeta)}$ 有两个孩子节点 $v_{2j-1}^{(\zeta+1)}, v_{2j}^{(\zeta+1)}$ 位于第 $(\zeta+1)$ 层，分别对应于集合 $V_{2j-1}^{(\zeta+1)}, V_{2j}^{(\zeta+1)}$，其中 $V_{2j-1}^{(\zeta+1)}$ 是 $V_j^{(\zeta)}$ 的较小一半，而 $V_{2j}^{(\zeta+1)}$ 是较大一半。因此，

$$V_j^{(\zeta)} = \{i \in \mathbb{Z} : (j-1)2^{q-\zeta} < i \le j2^{q-\zeta}\}.$$

树中的每个节点 $v_j^{(\zeta)}$ 存储一个和 $s_j^{(\zeta)} := \sum_{i \in V_j^{(\zeta)}} s_i$。

该数据结构支持附录 C 中所需的四种类型的操作，如下所示：

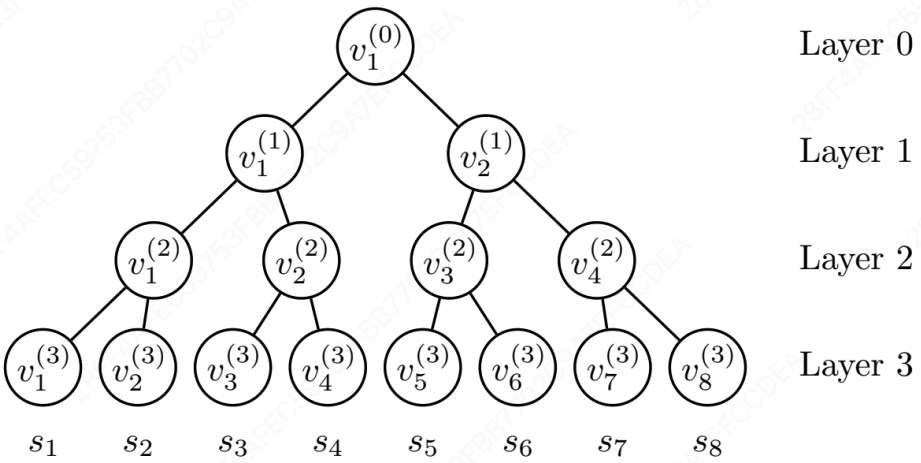


图 5：$q = 3$ 时的基本二叉树数据结构示例。（注：此处为图注翻译，具体图形请参考原文）

1. **Initialize(a)：** 在前半部分 $(a_1, \ldots, a_{n/2})$ 和后半部分 $(a_{n/2+1}, \ldots, a_n)$ 上递归运行 Initialize，以获得以 $v_1^{(1)}$ 和 $v_2^{(1)}$ 为根的两个子树。然后添加一个根 $v_1^{(0)}$，存储 $s_1^{(0)} \leftarrow s_1^{(1)} + s_2^{(1)}$。
2. **Sum(S)：** 简单地输出 $s_1^{(0)}$。
3. **Find(S, r)：** 如果 $r < s_1^{(1)}$，在以 $v_1^{(1)}$ 为根的左子树上递归调用 Find。否则，在以 $v_2^{(1)}$ 为根的右子树上递归调用 Find，并将 r 替换为 $r - s_1^{(1)}$。一旦到达叶子节点 $v_\ell^{(q)}$，返回 ℓ。
4. **Update(S, a, i_1, i_2)：** 如果 $i_2 \le n/2$，在以 $v_1^{(1)}$ 为根的左子树上递归调用 Update。如果 $i_1 > n/2$，在以 $v_2^{(1)}$ 为根的右子树上递归调用 Update。否则，我们有 $i_1 \le n/2 < i_2$，我们在左子树上调用 Update 处理索引 $i_1, n/2$，并在右子树上调用 Update 处理索引 $n/2+1, i_2$。在所有情况下，我们最后更新 $s_1^{(0)} \leftarrow s_1^{(1)} + s_2^{(1)}$。运行时间与我们更新的节点数成正比。仅当 $V_j^{(\zeta)} \cap \{i_1, \ldots, i_2\} \ne \emptyset$ 时，我们需要更新存储在 $v_j^{(\zeta)}$ 处的 $s_j^{(\zeta)}$。对于每个 ζ，这样的 j 的数量至多为 $(i_2 - i_1 + 1)/2^{q-\zeta} + 2$。对 $\zeta = 0, \ldots, q$ 求和，我们需要更新的节点总数为 $O((i_2 - i_1 + 1) + q) = O((i_2 - i_1 + 1) + \log n)$。

E. 额外实验与数据 (Additional Experiments and Data)

在本节中，我们提供了第 3 节中执行的实验的加速比和绝对运行时间表。此外，我们提供了定理 2.3 中描述的算法方法的实验数据。

E.1 改进的近似比 (Improved Approximation Ratio)

实验设置： 本实验旨在比较定理 2.3 中概述的算法方法与第 3.2 节中所做的直接使用 PRONE 作为聚类算法的方法。为此，我们使用 PRONE 作为敏感度采样的近似算法，然后使用加权变体的 k-means++ 算法对核心集进行聚类。这种方法在本节其余部分称为 **PRONE (boosted)**。该流程需要参数：中心数量 k 和指示敏感度采样产生的核心集大小的超参数 α。我们旨在比较我们的方法与 k-means++ 的聚类代价（见定义 A.1）和运行时间。

我们在第 3 节描述的数据集上运行这两种算法，并选择 $k \in \{10, 25, 50, 100, 250, 500, 1000, 2500, 5000\}$ 和 $\alpha \in \{0.001, 0.01, 0.1\}$。每种算法运行 5 次。

改进近似比代价的结果： 图 6 显示了该算法产生的中心代价相对于 k-means++ 产生的中心代价。它还包含 PRONE 相对于 k-means++ 的 $(k, 2)$-聚类代价。我们可以看到，在所有真实数据集上，只要 $\alpha n \ll k$，PRONE (boosted) 产生的解的质量与 k-means++ 相同或更好。这表明虽然 PRONE 本身产生的中心质量较差，但 PRONE (boosted) 变体产生的中心质量与普通 k-means++ 相当。当 $\alpha n \approx k$ 时，我们在 KDD、Song 和 Gaussian 数据集的图中观察到对应于 $\alpha \in \{0.1, 0.01\}$ 的线条末端有代价上升。

Boosted 方法优于 PRONE（PRONE 通常比其他算法差一个常数因子），并且有助于显著减少解质量的方差。在 Gaussian 数据集上，我们观察到未能从中心簇采样点的情况，这解释了 $\alpha = 0.1$ 对应的线条在 $k = 2500$ 处的尖峰。

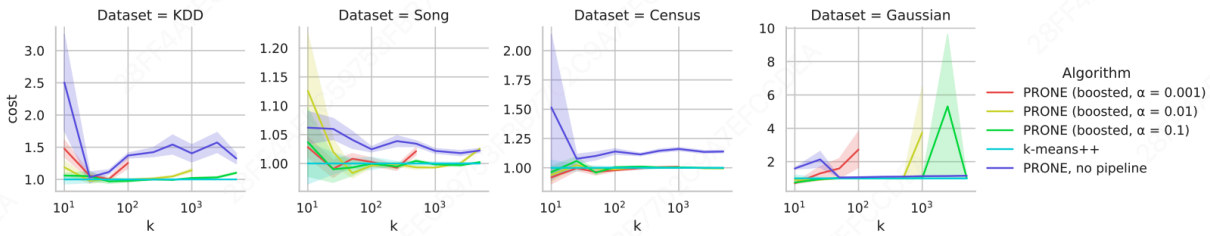


图 6： Boosted 变体与 k-means++ 相比的聚类代价。图中的线显示了 Boosted 算法产生的中心代价相对于 k-means++ 的比值，中心数量范围从 10 到 5000。深蓝色表示非 Boosted 版本。

运行时间： 表 2 显示了 Boosted 方法相对于使用普通 k-means++ 计算中心所需时间的加速比。我们算法的运行时间现在随 k 扩展，但与 k-means++ 相比速度较慢，因为我们必须在更小的数据集上运行它。再一次，我们观察到显著的加速，特别是随着 k 变大。

- 正如预期的那样，加速比取决于超参数 α 的选择。随着 k 的扩展，我们观察到较大 α 的收益递减，对于 $k \geq 100$，除 Gaussian 外，各数据集的加速比基本保持恒定。这是由于算法的运行时间由在核心集上执行 k-means++ 的时间主导，其渐近运行时间为 $O(\alpha ndk)$。使用这种方法我们可以实现的加速是显著的，比 k-means++ 快高达 118 倍。我们预计在海量数据集上，可以实现更大的加速。
- 有趣的是，加速比可以非常接近甚至超过比例 α，如在 KDD 和 Song 数据集上观察到的那样。Boosted 方法的最后阶段在大小为 αn 的核心集上执行 k-means++，因此该步骤的运行时间应为 $O(\alpha ndk)$。观察到的额外加速可能是由于算法的 k-means++ 步骤中更好的缓存和内存利用率。

Dataset	Centers Algorithm	10	25	50	100	250	500	1000	2500	5000
Census	PRONE (boosted, $\alpha = 0.001$)	1.6	4.0	7.9	15.6	36.8	69.5	118.5	-	-
	PRONE (boosted, $\alpha = 0.01$)	1.5	3.7	6.7	11.7	20.8	28.5	30.1	36.2	41.9
	PRONE (boosted, $\alpha = 0.1$)	1.0	1.8	2.3	2.7	3.0	3.2	3.0	3.0	3.2
	k-means++	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
Song	PRONE (boosted, $\alpha = 0.001$)	2.1	5.3	10.7	21.3	51.7	97.5	-	-	-
	PRONE (boosted, $\alpha = 0.01$)	2.1	5.0	9.8	18.5	38.0	59.1	81.5	108.3	117.0
	PRONE (boosted, $\alpha = 0.1$)	1.3	2.2	2.8	3.4	3.8	4.0	4.0	4.0	3.9
	k-means++	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
KDD	PRONE (boosted, $\alpha = 0.001$)	1.8	5.0	9.6	20.6	-	-	-	-	-
	PRONE (boosted, $\alpha = 0.01$)	1.9	4.8	9.0	17.5	37.1	59.2	92.5	-	-
	PRONE (boosted, $\alpha = 0.1$)	1.4	2.7	3.6	4.9	5.7	6.2	7.1	6.6	6.4
	k-means++	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
Gaussian	PRONE (boosted, $\alpha = 0.001$)	0.5	0.9	1.9	3.6	-	-	-	-	-
	PRONE (boosted, $\alpha = 0.01$)	0.5	1.0	2.0	3.4	8.2	14.8	25.6	-	-
	PRONE (boosted, $\alpha = 0.1$)	0.4	0.8	1.4	2.2	4.0	5.0	6.1	6.8	7.2
	k-means++	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0

表 2： 计算不同数据集的聚类和分配时相对于 k-means++ 的平均加速比。换句话说，每个单元格包含 $T_{k\text{-means++}}/T_{\text{PRONE}}$。缺失的条目表示 $\alpha n > k$ 的情况。

E.2 额外数据 (Additional Data)

在本节中，我们为第 3.2 节中执行的实验提供全范围参数的运行时间数据。表 3 显示了相对于 k-means++ 的加速比，类似于表 1 中的右侧表格。此外，表 4 提供了以毫秒为单位的绝对运行时间。

Dataset	Centers Algorithm	10	25	50	100	250	500	1000	2500	5000
Census	PRONE	1.5	3.8	7.5	15.1	36.2	73.2	142.2	351.9	662.5
	PRONE (variance)	0.5	1.1	2.2	4.6	11.0	22.2	43.7	109.5	214.7
	PRONE (covariance)	0.2	0.5	1.1	2.2	5.2	10.7	21.0	54.7	117.4
	k-means++	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
Song	PRONE	2.0	5.0	9.7	19.1	46.1	95.5	188.2	443.0	837.5
	PRONE (variance)	0.5	1.1	2.3	4.5	11.4	23.1	44.2	110.9	217.2
	PRONE (covariance)	0.2	0.4	0.8	1.5	4.0	8.2	15.5	40.0	82.4
	k-means++	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
KDD	PRONE	1.5	3.7	6.9	16.3	39.5	68.3	158.5	414.7	727.5
	PRONE (variance)	0.7	1.7	3.1	6.3	16.1	32.0	63.4	159.6	312.4
	PRONE (covariance)	0.3	0.7	1.3	2.6	6.8	12.9	25.8	58.5	128.4
	k-means++	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
Gaussian	PRONE	0.5	1.0	1.9	3.8	9.2	18.3	35.7	85.3	165.9
	PRONE (variance)	0.5	1.0	2.0	3.8	9.1	17.7	34.5	83.6	162.9
	PRONE (covariance)	0.4	1.0	1.7	3.6	8.2	16.1	31.8	79.9	152.6
	k-means++	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0

表 3： 计算不同数据集的聚类和分配时相对于 k-means++ 的平均加速比。换句话说，每个单元格包含 $T_{k\text{-means++}}/T_{\text{PRONE}}$。

Dataset	Centers Algorithm	10	25	50	100	250	500	1000	2500	5000
Census	PRONE	525.1 ± 15.3	534.3 ± 23.4	534.6 ± 20.8	530.1 ± 15.4	538.9 ± 15.2	533.9 ± 11.7	547.4 ± 23.8	548.2 ± 13.5	563.7 ± 8.7
	PRONE (variance)	1750.6 ± 71.7	1778.9 ± 38.1	1780.3 ± 23.9	1752.7 ± 56.9	1768.6 ± 28.2	1757.6 ± 37.8	1778.9 ± 27.5	1761.0 ± 73.3	1739.6 ± 10.9
	PRONE (covariance)	3769.0 ± 152.2	3882.5 ± 167.8	3743.9 ± 316.7	3714.2 ± 378.3	3766.4 ± 220.2	3662.9 ± 122.6	3708.7 ± 372.4	3525.6 ± 609.9	3182.2 ± 288.7
	k-means++	812.5 ± 20.9	2040.6 ± 25.6	3994.3 ± 25.3	7992.9 ± 91.5	19519.0 ± 161.5	39058.4 ± 337.8	77823.6 ± 527.5	192913.1 ± 3657.0	373488.2 ± 221.9
Song	PRONE	104.4 ± 5.5	101.8 ± 5.2	106.5 ± 4.1	109.9 ± 6.8	113.7 ± 8.8	109.0 ± 6.4	108.7 ± 3.9	117.2 ± 6.3	120.8 ± 12.9
	PRONE (variance)	443.3 ± 5.0	448.3 ± 8.0	450.1 ± 9.9	468.0 ± 31.6	458.8 ± 15.1	450.7 ± 10.0	462.6 ± 11.6	468.3 ± 14.0	465.7 ± 14.9
	PRONE (covariance)	1164.4 ± 162.1	1266.5 ± 136.6	1332.4 ± 116.3	1381.8 ± 108.4	1322.2 ± 173.4	1263.0 ± 175.6	1324.2 ± 108.4	1296.8 ± 177.8	1228.0 ± 175.2
	k-means++	207.4 ± 4.0	513.7 ± 9.8	1036.6 ± 17.0	2103.9 ± 44.2	5245.9 ± 143.6	10412.5 ± 119.3	20464.1 ± 196.9	51917.4 ± 554.3	101149.0 ± 1962.7
KDD	PRONE	31.2 ± 5.2	32.2 ± 7.4	34.1 ± 1.8	28.9 ± 4.9	29.2 ± 9.1	34.6 ± 6.3	30.8 ± 8.6	28.8 ± 5.2	33.4 ± 6.6
	PRONE (variance)	72.0 ± 1.4	71.8 ± 1.4	76.5 ± 7.2	75.1 ± 7.1	71.8 ± 1.2	73.9 ± 2.1	77.0 ± 4.6	74.7 ± 3.2	77.7 ± 5.9
	PRONE (covariance)	169.3 ± 2.9	173.5 ± 5.5	180.3 ± 13.1	184.1 ± 21.2	170.8 ± 8.9	182.6 ± 19.0	188.7 ± 16.9	204.1 ± 40.3	189.1 ± 11.6
	k-means++	48.0 ± 0.3	119.3 ± 3.7	233.9 ± 3.2	470.4 ± 11.6	1153.5 ± 2.1	2363.3 ± 108.6	4878.1 ± 171.1	11928.9 ± 234.3	24285.9 ± 243.8
Gaussian	PRONE	30.0 ± 2.3	29.1 ± 2.2	30.6 ± 3.3	29.1 ± 1.4	29.1 ± 1.0	28.7 ± 0.2	29.3 ± 0.4	30.2 ± 0.6	31.5 ± 0.6
	PRONE (variance)	29.8 ± 3.0	28.9 ± 0.3	29.5 ± 1.0	29.6 ± 0.6	29.6 ± 0.3	29.6 ± 0.4	30.2 ± 0.3	30.8 ± 0.2	32.1 ± 0.6
	PRONE (covariance)	31.8 ± 2.2	31.5 ± 2.1	34.6 ± 6.6	30.8 ± 0.3	32.7 ± 2.8	32.6 ± 2.4	32.8 ± 2.3	32.2 ± 0.5	34.3 ± 0.4
	k-means++	13.7 ± 2.2	30.0 ± 0.4	59.5 ± 2.2	111.5 ± 2.8	268.6 ± 1.5	525.3 ± 1.9	1043.3 ± 1.4	2575.3 ± 5.0	5229.6 ± 135.0

表 4： 计算不同数据集的聚类和分配时的平均运行时间（毫秒）和标准差，相对于 k-means++。